학습 관련 기술

매개변수 갱신 · 가중치의 초깃값 · 배치 정규화 · 가중치감소 · 드롭아웃 · 하이퍼파라미터

SGD → Momentum → AdaGrad → RMSProp → Adam → Xavier · He 초기화 → Batch Norm → Weight Decay · AdamW · Dropout

이번 주의 핵심 질문

지난주에는 역전파로 손실의 gradient $\frac{\partial L}{\partial W}$를 계산하고,

$$W \leftarrow W - \alpha \frac{\partial L}{\partial W}$$

로 매개변수를 갱신하여 MNIST를 학습시켰다. 그런데 이 갱신 규칙이 최선일까? gradient가 가리키는 방향이 늘 최솟값을 향하는 것은 아니고, 학습률 α 하나로 모든 매개변수를 같은 비율로 움직이는 것도 비효율적일 수 있다. **같은 gradient 정보를 쓰되, 더 빠르고 안정적으로 최솟값을 찾아가는 방법**은 무엇일까?

학습목표

이 주차가 끝나면 다음을 설명하고 직접 계산할 수 있어야 한다.

1. SGD, Momentum, AdaGrad, RMSProp, Adam, AdamW의 원리를 설명한다.
2. 각 최적화 기법을 `update(params, grads)` 형태의 코드로 구현한다.
3. 초기 가중치가 은닉층의 활성화값 분포와 학습에 주는 영향(기울기 소실, 표현력 제한)을 설명한다.
4. Xavier 초깃값과 He 초깃값을 설명하고 구현한다.
5. 배치 정규화의 순전파 식과 계산 그래프를 이해하고, 역전파를 유도하여 구현한다.
6. 오버피팅을 훈련 정확도와 시험(검증) 정확도의 차이로 확인하고, 그 원인을 설명한다.
7. 가중치 감소(L2 규제, AdamW)와 드롭아웃의 원리를 설명하고 구현한다.
8. 로그 눈금의 무작위 탐색으로 최적의 하이퍼파라미터를 찾는다.

1 매개변수 갱신: 최적화 문제

1.1 신경망 학습은 최적화 문제이다

신경망의 학습은 손실함수 $L(W)$를 가장 작게 만드는 매개변수 W를 찾는 문제, 즉 **최적화(optimization)** 문제이다. 매개변수의 개수는 MNIST의 작은 network만 해도 수만 개이고, 손실함수의 모양은 매우 복잡하여 최솟값을 식으로 한 번에 구할 수 없다. 그래서 지금까지 우리는 gradient를 단서로 하여 조금씩 매개변수를 고쳐 나가는 방법을 썼다. 이번 주에는 이 **갱신 규칙**을 개선하는 방법들을 살펴본다.

이 절 전체에서 다음 기호를 사용한다.

기호	의미
W	갱신할 매개변수 (행렬이어도 된다). w_i는 그 i번째 성분, 즉 매개변수 하나
$\boldsymbol{g}_k = \frac{\partial L}{\partial W}\Big\vert_{W=W_k}$	k번째 단계의 gradient. 역전파로 구한 $\frac{\partial L}{\partial W}$이며 W와 같은 shape이다. 성분 $g_{k,i} = \frac{\partial L}{\partial w_i}$는 매개변수 w_i 하나의 기울기
$\boldsymbol{v}_k,\ \boldsymbol{h}_k,\ \boldsymbol{m}_k$	기법마다 기억해 두는 양. 모두 W와 같은 shape의 벡터(배열)이다
α	학습률 (learning rate)
$\odot, \sqrt{\cdot}, \frac{\cdot}{\cdot}, \lvert\cdot\rvert$	성분별 곱, 성분별 제곱근, 성분별 나눗셈, 성분별 절댓값

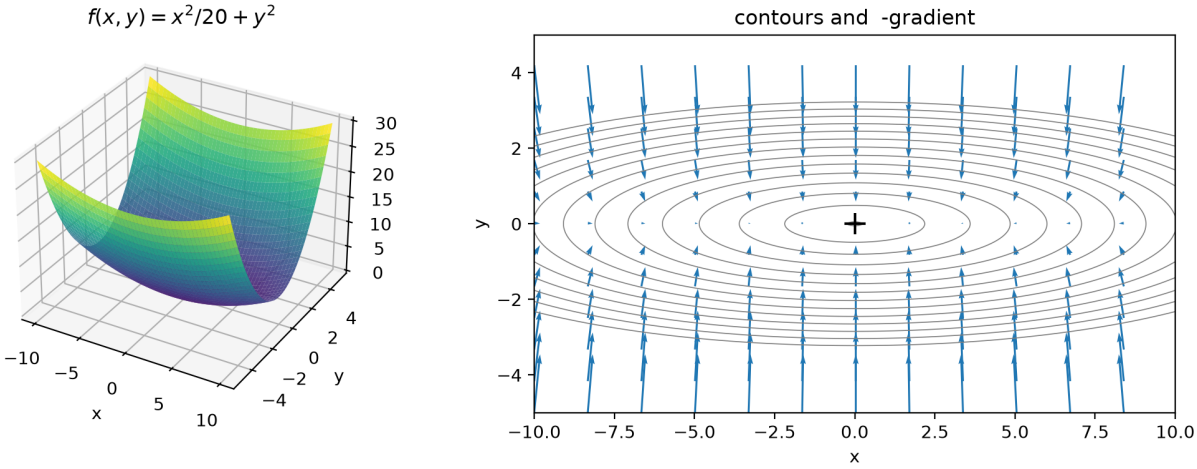
굵은 글씨($\boldsymbol{g}_k, \boldsymbol{v}_k, \boldsymbol{h}_k, \ldots$)는 매개변수 전체와 같은 크기의 벡터이고, 보통 글씨에 첨자 i를 붙인 $g_{k,i}$, $h_{k,i}$는 그 i번째 성분(스칼라)이다. 매개변수 W는 지금까지처럼 대문자로 쓴다. 코드에서 $\boldsymbol{g}_k$는 역전파가 돌려준 `grads[key]`이다.

1.2 예제 함수: $f(x,y) = \frac{1}{20}x^2 + y^2$

여러 기법을 눈으로 비교하기 위해, 매개변수가 두 개뿐인 다음 함수의 최솟값을 찾는 문제를 생각하자.

$$\boxed{f(x,y) = \frac{1}{20}x^2 + y^2, \qquad \nabla f(x,y) = \left(\frac{x}{10},\ 2y\right)}$$

최솟값은 $(0,0)$에서 0이다. 시작점은 모든 기법에서 $(x_0, y_0) = (-7,\ 2)$로 같게 둔다.



$f(x,y) = x^2/20 + y^2$의 그래프(왼쪽)와 등고선 및 $-\nabla f$의 방향(오른쪽)

이 함수는 x 방향으로 아주 완만하고 y 방향으로 가파른 **길고 좁은 골짜기** 모양이다. 두 방향의 곡률(이계도함수)은

$$\frac{\partial^2 f}{\partial x^2} = \frac{1}{10}, \qquad \frac{\partial^2 f}{\partial y^2} = 2$$

로 20배나 차이가 난다. 그래서 오른쪽 그림처럼 대부분의 점에서 $-\nabla f$는 최솟값 $(0,0)$이 아니라 거의 y축 방향(골짜기의 벽 쪽)을 가리킨다. 실제 딥러닝의 손실함수에서도 이처럼 방향마다 곡률이 크게 다른 경우가 흔하다.

이 예제에서 각 기법은 모두 같은 일을 반복한다. 현재 위치의 gradient $\boldsymbol{g}_k = \nabla f(x_k, y_k)$를 계산하고 (신경망에서는 이것이 $\frac{\partial L}{\partial W}$이다. 예제에서는 매개변수가 $W = (x, y)$ 두 개뿐이다), 갱신 규칙에

따라 다음 위치 (x_{k+1}, y_{k+1})로 이동한다. **달라지는 것은 오직 갱신 규칙뿐**이다.

1.3 코드의 구조

모든 최적화 기법을 같은 모양의 클래스로 만든다. 매개변수와 gradient를 같은 key를 가진 dictionary로 받아, `update` 메서드가 매개변수를 **제자리에서** 고친다.

```
def f(x, y):
    return x ** 2 / 20.0 + y ** 2

def grad_f(x, y):
    return x / 10.0, 2.0 * y

def minimize(optimizer, start=(-7.0, 2.0), steps=30):
    """Run `steps` updates from `start`; return the path as an array of shape (steps+1, 2)."""
    params = {"x": np.array(start[0]), "y": np.array(start[1])}
    path = [(float(params["x"]), float(params["y"]))]
    for _ in range(steps):
        gx, gy = grad_f(params["x"], params["y"])
        grads = {"x": gx, "y": gy}
        optimizer.update(params, grads)
        path.append((float(params["x"]), float(params["y"])))
    return np.array(path)
```

`params`, `grads`는 지난주 `mnist_lab.py`의 `network.params`, `grads`와 같은 형태이다. 딕셔너리 자료구조로서, 같은 키를 같고, 차원이 같은 `nd.array`를 갖고 있음을 기억하자. 실습에 사용한 $f(x, y)$에 대한 기울기는 `grad_f` 함수를 사용하여 구하지만, 실제 딥러닝에서 `grads`는 오차 역전파를 통해서 계산됨을 기억하자. 따라서 이번 주에 만드는 optimizer는 MNIST 학습 루프의

```
for key in network.params:
    network.params[key] -= learning_rate * grads[key]
```

부분을 `optimizer.update(network.params, grads)` 한 줄로 바꾸어 그대로 쓸 수 있다. `optimizer` class는 아래에서 정의한다.

코드를 모듈로 나누기. optimizer 클래스는 2변수 예제에서도, MNIST 학습에서도 똑같이 쓰인다. 그래서 같은 코드를 파일마다 복사하지 않고, 공통 부분을 **모듈**(별도의 `.py` 파일)로 저장해 두고 필요한 곳에서 `import`하여 쓴다. 이번 주 코드는 모두 `code` 폴더에 있으며 다음과 같이 구성되어 있다. 어느 위치에서 실행하든 그림은 강의 노트 옆의 `figures` 폴더에 저장된다.

파일	내용
`optimizers.py`	SGD, Momentum, AdaGrad, RMSProp, Adam 클래스
`layers.py`	Affine, ReLU, Sigmoid, softmax 손실 계층, `MultiLayerNet` (지난주 코드)
`mnist_data.py`	MNIST 불러오기, 정규화, one-hot, 훈련/검증 분할
`optimizers_2d.py`	2변수 예제 실험 (`optimizers.py` 사용)
`mnist_optimizer_compare.py`	MNIST 비교 실험 (1.9절, 세 모듈 모두 사용)

예를 들어 2변수 예제 코드의 첫머리는 다음과 같다.

```
import os
import numpy as np
import matplotlib.pyplot as plt

from optimizers import SGD, Momentum, AdaGrad, RMSProp, Adam   # optimizers.py
```

이후 각 기법을 설명할 때 보이는 클래스 코드는 모두 `optimizers.py`에 들어 있다.

1.4 확률적 경사하강법 (SGD)

갱신 규칙. 지금까지 사용한 방법이다.

$$\boxed{W_{k+1} = W_k - \alpha\, \boldsymbol{g}_k, \qquad \boldsymbol{g}_k = \left.\frac{\partial L}{\partial W}\right|_{W=W_k}}$$

성분으로 쓰면 $w_{k+1,i} = w_{k,i} - \alpha\, g_{k,i}$로, 모든 매개변수가 **같은 학습률** α로 움직인다. "**확률적**(stochastic)"이라는 이름은 $\boldsymbol{g}_k$를 전체 데이터가 아니라 **무작위로 고른 미니배치**의 평균 손실로 계산하기 때문에 붙었다 (지난주 미니배치 학습). 예제 함수에는 데이터가 없으므로 정확한 gradient를 쓰지만, 갱신 규칙은 같다.

```
class SGD:
    """W <- W - lr * dL/dW"""
    def __init__(self, lr=0.01):
        self.lr = lr

    def update(self, params, grads):
        for key in params:
            params[key] -= self.lr * grads[key]
```

예제에서의 움직임. $\nabla f = (x/10,\ 2y)$이므로 SGD의 갱신은 좌표마다 따로 쓸 수 있다.

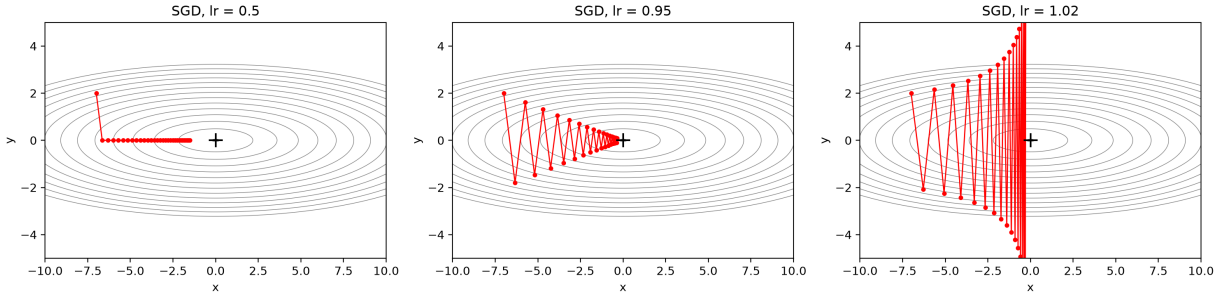
$$x_{k+1} = x_k - \alpha\frac{x_k}{10} = \Big(1 - \frac{\alpha}{10}\Big)x_k, \qquad y_{k+1} = y_k - \alpha\cdot 2y_k = (1-2\alpha)\,y_k$$

즉 매 단계 x는 $\big(1-\frac{\alpha}{10}\big)$배, y는 $(1-2\alpha)$배가 된다.

- y 방향이 발산하지 않으려면 $|1-2\alpha| < 1$, 즉 $\alpha < 1$이어야 한다.

- 그런데 $\alpha < 1$이면 x 방향의 비율은 $1 - \frac{\alpha}{10} > 0.9$로, x는 한 단계에 10%도 줄지 않는다.

곡률이 큰 방향(y) 때문에 학습률을 키울 수 없고, 그 작은 학습률로는 곡률이 작은 방향(x)이 아주 느리게 수렴하는 것이다.



학습률에 따른 SGD의 경로 (시작점 $(-7, 2)$, 30단계)

- $\alpha = 0.5$: y의 비율이 $1 - 2\alpha = 0$이라 한 번에 $y = 0$이 되지만, x는 매번 0.95배씩만 줄어 30단계 뒤에도 $x \approx -1.5$이다.
- $\alpha = 0.95$: y의 비율이 -0.9로 부호가 매번 바뀌어 골짜기 양쪽 벽을 오가며 **지그재그**로 움직인다.
- $\alpha = 1.02$: y의 비율이 $|1 - 2.04| = 1.04 > 1$이 되어 **발산**한다.

SGD의 약점

SGD의 이동 방향은 언제나 현재 위치의 $-\boldsymbol{g}_k$이다. 그런데 방향마다 곡률이 다른 함수에서는 $-\boldsymbol{g}_k$가 최솟값을 향하지 않는다. 게다가 학습률 α 하나가 모든 방향의 보폭을 정하므로, 가파른 방향에 맞추면 완만한 방향이 느리고, 완만한 방향에 맞추면 가파른 방향이 발산한다. 이후의 기법들은 이 두 문제, 즉 **방향의 문제**와 **보폭의 문제**를 각각 개선한다.

1.5 모멘텀 (Momentum)

아이디어: 공을 굴린다. 경사면에 공을 놓으면, 공은 그 순간의 경사만 따라 움직이지 않는다. 이미 가지고 있는 **속도**가 있어서, 같은 방향으로 계속 내려가면 점점 빨라지고 방향이 자꾸 바뀌는 쪽의 움직임은 서로 상쇄된다. 모멘텀은 이 **속도** $\boldsymbol{v}$를 도입한다. 속도도 매개변수마다 하나씩 있으므로 W와 같은 shape의 벡터이다.

갱신 규칙.

$$\boxed{\boldsymbol{v}_{k+1} = \beta\,\boldsymbol{v}_k - \alpha\,\boldsymbol{g}_k, \qquad W_{k+1} = W_k + \boldsymbol{v}_{k+1}} \qquad (\boldsymbol{v}_0 = \boldsymbol{0})$$

$\beta \in [0, 1)$ (보통 0.9)는 이전 속도를 얼마나 유지할지를 정하며, 마찰의 역할을 한다. $\beta = 0$이면 SGD와 같다.

속도는 과거 gradient의 가중 합이다. 점화식을 풀면

$$\boldsymbol{v}_{k+1} = -\alpha\left(\boldsymbol{g}_k + \beta\,\boldsymbol{g}_{k-1} + \beta^2\boldsymbol{g}_{k-2} + \cdots + \beta^k\boldsymbol{g}_0\right) = -\alpha\sum_{j=0}^{k}\beta^{k-j}\boldsymbol{g}_j$$

이다. 즉 모멘텀은 **최근 gradient 일수록 크게, 오래된 gradient 일수록 작게** 더한 방향으로 움직인다.

- 예제의 y 방향처럼 gradient의 부호가 매번 바뀌는 방향에서는 더한 값이 서로 **상쇄**되어 진동이 줄어든다.
- x 방향처럼 gradient가 계속 같은 부호인 방향에서는 더한 값이 **누적**되어 점점 빨라진다. 같은 gradient $\boldsymbol{g}$가 계속되면 속도는 $-\alpha(1+\beta+\beta^2+\cdots)\boldsymbol{g} \to -\dfrac{\alpha}{1-\beta}\boldsymbol{g}$이므로, $\beta = 0.9$일 때 보폭이 최대 10배까지 커진다.

```
class Momentum:
    """v <- beta * v - lr * dL/dW,   W <- W + v"""
    def __init__(self, lr=0.01, beta=0.9):
        self.lr = lr
        self.beta = beta
        self.v = None

    def update(self, params, grads):
        if self.v is None:                          # velocity starts at 0
            self.v = {key: np.zeros_like(val) for key, val in params.items()}
        for key in params:
            self.v[key] = self.beta * self.v[key] - self.lr * grads[key]
            params[key] += self.v[key]
```

예제: 첫 두 단계. $\alpha = 0.1, \beta = 0.9, (x_0, y_0) = (-7, 2)$이면 $\boldsymbol{g}_0 = (-0.7,\ 4)$이므로

$$\boldsymbol{v}_1 = -0.1\,(-0.7,\ 4) = (0.07,\ -0.4), \qquad (x_1, y_1) = (-6.93,\ 1.6).$$

다음 단계에서 $\boldsymbol{g}_1 = (-0.693,\ 3.2)$이므로

$$\boldsymbol{v}_2 = 0.9\,(0.07,\ -0.4) - 0.1\,(-0.693,\ 3.2) = (0.1323,\ -0.68), \qquad (x_2, y_2) = (-6.7977,\ 0.92).$$

x 방향의 속도가 $0.07 \to 0.1323$으로 거의 두 배가 되었다. 속도가 쌓이고 있는 것이다.

1.6 AdaGrad

아이디어: 매개변수마다 학습률을 다르게. SGD의 두 번째 문제는 학습률 하나가 모든 방향의 보폭을 정한다는 것이었다. AdaGrad(Adaptive Gradient)는 지금까지 **gradient가 크게 나왔던 매개변수는 학습률을 줄이고,** 작게 나왔던 매개변수는 상대적으로 크게 유지한다.

갱신 규칙.

$$\boxed{\boldsymbol{h}_{k+1} = \boldsymbol{h}_k + \boldsymbol{g}_k \odot \boldsymbol{g}_k, \qquad W_{k+1} = W_k - \alpha\,\frac{\boldsymbol{g}_k}{\sqrt{\boldsymbol{h}_{k+1}} + \varepsilon}} \qquad (\boldsymbol{h}_0 = \boldsymbol{0})$$

$\boldsymbol{h}$도 W와 같은 shape의 벡터이며, 제곱, 제곱근, 나눗셈은 모두 **성분별**이다. ε (예: 10^{-7})은 0으로 나누는 것을 막는 작은 수이다.

성분으로 쓰면. 매개변수 w_i 하나에 대해서만 쓰면 다음과 같다.

$$h_{k+1,i} = h_{k,i} + g_{k,i}^2 = g_{0,i}^2 + g_{1,i}^2 + \cdots + g_{k,i}^2, \qquad w_{k+1,i} = w_{k,i} - \frac{\alpha}{\sqrt{h_{k+1,i}} + \varepsilon}\, g_{k,i}$$

즉 매개변수 w_i의 실제 학습률은

$$\alpha_{k,i} = \frac{\alpha}{\sqrt{g_{0,i}^2 + g_{1,i}^2 + \cdots + g_{k,i}^2} + \varepsilon}$$

이다. **그 매개변수 자신의** 과거 gradient 제곱의 합으로 나누므로, gradient가 컸던 매개변수일수록 보폭이 작아지고, 작았던 매개변수는 보폭이 상대적으로 크게 유지된다. 매개변수마다 서로 다른 학습률을 쓰는 것이다.

예제에서의 효과. 첫 단계에서는 $\boldsymbol{h}_1 = \boldsymbol{g}_0 \odot \boldsymbol{g}_0$, 즉 $h_{1,i} = g_{0,i}^2$이므로 (ε은 무시)

$$W_1 = W_0 - \alpha\, \frac{\boldsymbol{g}_0}{|\boldsymbol{g}_0|}, \qquad \text{성분으로는} \qquad w_{1,i} = w_{0,i} - \alpha\, \frac{g_{0,i}}{|g_{0,i}|} = w_{0,i} - \alpha\, \mathrm{sign}(g_{0,i})$$

즉 **각 좌표가 gradient의 크기와 관계없이 정확히** α**만큼** 움직인다. $\alpha = 1.5$, $\boldsymbol{g}_0 = (-0.7,\ 4)$이면 x의 실제 학습률은 $\frac{1.5}{0.7}$, y의 실제 학습률은 $\frac{1.5}{4}$로 서로 다르고,

$$(x_1, y_1) = (-7 + 1.5,\ 2 - 1.5) = (-5.5,\ 0.5)$$

이다. SGD에서는 x 방향 gradient가 y 방향의 $\frac{1}{20}$도 안 되어 x가 거의 움직이지 않았지만, AdaGrad는 두 방향의 보폭을 비슷한 수준으로 맞추어 준다. 곡률의 차이를 학습률로 보정하는 셈이다 (Newton's method가 H^{-1}로 방향마다 보폭을 조절했던 것과 비슷한 효과이다).

두 가지로 읽는 같은 식

매개변수 w_i 하나의 갱신량은 다음 두 가지로 묶어 읽을 수 있다.

$$\Delta w_i = -\underbrace{\frac{\alpha}{\sqrt{h_{k+1,i}}+\varepsilon}}_{\text{매개변수별 학습률}} \; g_{k,i} = -\alpha \underbrace{\frac{g_{k,i}}{\sqrt{h_{k+1,i}}+\varepsilon}}_{\text{다시 스케일한 gradient}}$$

- **매개변수별 학습률로 보면**: gradient가 컸던 매개변수는 작은 학습률로 조심스럽게, 작았던 매개변수는 큰 학습률로 과감하게 움직인다. $h_{k,i}$가 커지기만 하여 학습률이 0으로 줄어드는 AdaGrad의 약점도 이 관점에서 자연스럽게 보인다.
- **다시 스케일한 gradient로 보면**: gradient의 성분마다 다른 수로 나누므로 이동 방향이 더 이상 $-\boldsymbol{g}_k$와 평행하지 않다. 즉 가장 가파른 방향으로 가지 않는다. 위 예제에서 x 방향 gradient가 y 방향의 $\frac{1}{20}$도 안 되는데도 x가 y와 같은 거리를 움직인 것이 그 예이다. 또 $h_{k+1,i}$에는 현재의 $g_{k,i}^2$도 들어 있으므로, "학습률"은 미리 정해진 값이 아니라 현재 gradient에 따라서도 달라진다 (첫 단계의 갱신량은 $-\alpha\,\mathrm{sign}(g_{0,i})$였다).

벡터로 쓰면 두 관점이 하나로 묶인다.

$$W_{k+1} = W_k - \alpha\, D_k^{-1}\boldsymbol{g}_k, \qquad D_k = \mathrm{diag}\big(\sqrt{\boldsymbol{h}_{k+1}}+\varepsilon\big)$$

대각행렬 D_k^{-1}을 α 쪽에 붙이면 매개변수별 학습률이고, $\boldsymbol{g}_k$ 쪽에 붙이면 다시 스케일한 gradient이다. Newton 방법이 $H^{-1}\boldsymbol{g}$로 방향마다 보폭을 조절했듯이, AdaGrad (그리고 RMSProp, Adam)는 H^{-1} 대신 계산이 값싼 **대각행렬** D^{-1}을 쓰는 셈이다. 이것을 **대각 전처리(diagonal preconditioning)**라고 한다.

```python
class AdaGrad:
    """h <- h + g*g,   W <- W - lr * g / (sqrt(h) + eps)"""
    def __init__(self, lr=0.01, eps=1e-7):
        self.lr = lr
        self.eps = eps
        self.h = None

    def update(self, params, grads):
        if self.h is None:
            self.h = {key: np.zeros_like(val) for key, val in params.items()}
        for key in params:
            self.h[key] += grads[key] * grads[key]           # sum of squared gradients
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + self.eps)
```

AdaGrad의 약점과 RMSProp. $h_{k,i}$는 제곱을 **계속 더하기만** 하므로 학습이 진행될수록 커지기만 하고, 실제 학습률 $\alpha/\sqrt{h_{k,i}}$는 계속 작아져 결국 거의 움직이지 않게 된다. **RMSProp**은 과거 gradient를 모두 똑같이 더하는 대신 **지수이동평균**으로 오래된 것을 서서히 잊는다.

$$\boxed{\boldsymbol{h}_{k+1} = \rho\,\boldsymbol{h}_k + (1-\rho)\,\boldsymbol{g}_k \odot \boldsymbol{g}_k, \qquad W_{k+1} = W_k - \alpha\,\frac{\boldsymbol{g}_k}{\sqrt{\boldsymbol{h}_{k+1}}+\varepsilon}} \qquad (\rho \approx 0.9)$$

성분으로는 $h_{k+1,i} = \rho\, h_{k,i} + (1-\rho)\, g_{k,i}^2$이다.

지수이동평균. 점화식을 풀면 ($\boldsymbol{h}_0 = \mathbf{0}$)

$$h_{k+1,i} = (1-\rho)\big(g_{k,i}^2 + \rho\, g_{k-1,i}^2 + \rho^2 g_{k-2,i}^2 + \cdots + \rho^k g_{0,i}^2\big) = \sum_{j=0}^{k} (1-\rho)\rho^{k-j}\, g_{j,i}^2$$

이다. $\rho = 0.9$이면 가중치는 최근 것부터 0.1, 0.09, 0.081, ...이다.

- **평균**: 가중치의 합이 $(1-\rho)(1+\rho+\cdots+\rho^k) = 1-\rho^{k+1} \to 1$이므로 가중평균이다.
- **지수**: 오래된 gradient일수록 가중치가 ρ^{k-j}로 지수적으로 작아져 서서히 잊힌다.
- **이동**: 단계가 진행되면 평균을 내는 구간도 함께 이동한다. 실질적으로 최근 약 $1/(1-\rho)$단계 ($\rho = 0.9$이면 약 10단계)의 평균이다.

따라서 $h_{k,i}$는 매개변수 w_i의 최근 gradient 제곱의 **평균** 정도로 유지되므로 학습률이 0으로 줄어들지 않는다. (모멘텀의 $\boldsymbol{v}$도 같은 모양의 합이지만, $1-\beta$를 곱하지 않으므로 평균이 아니라 누적합이다.)

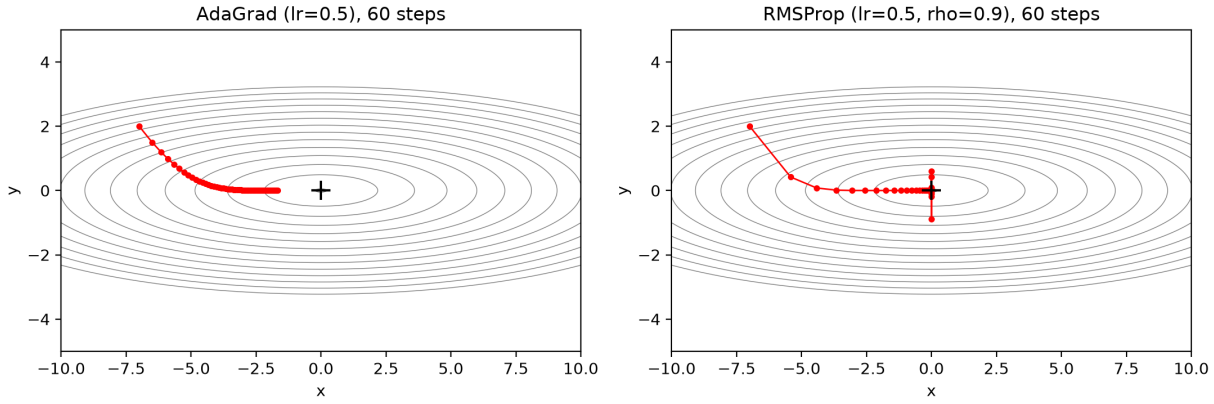
이름의 뜻: Root Mean Square. RMSProp은 **Root Mean Square Propagation**의 약자이다. 숫자들의 **RMS**(제곱평균제곱근)는 제곱하고, 평균을 내고, 제곱근을 씌운 값 $\sqrt{\frac{1}{n}\sum_j x_j^2}$으로, 부호와 관계없이 그 숫자들이 보통 얼마나 큰지를 나타낸다. RMSProp에서 $\sqrt{h_{k,i}}$가 바로 매개변수 w_i의 **최근 gradient들의 RMS**이므로,

$$\Delta w_i = -\alpha\, \frac{g_{k,i}}{\sqrt{h_{k+1,i}} + \varepsilon} \approx -\alpha\, \frac{g_{k,i}}{\text{최근 } g_{\cdot,i}\text{의 RMS}}$$

즉 gradient를 그 매개변수의 gradient가 보통 갖는 크기로 나누어, 크기를 1 정도로 맞춘다. 예를 들어 gradient가 계속 같은 값 g이면 $h \to g^2$이므로 갱신량은 $-\alpha\,\mathrm{sign}(g)$가 된다. AdaGrad는 평균이 아니라 **합**의 제곱근 $\sqrt{\sum_j g_{j,i}^2}$을 쓰므로 단계가 늘수록 커지기만 하지만, RMSProp은 이것을 평균으로 바꾸어 일정한 수준에 머물게 한 것이다. 이름의 "Prop"은 전체 데이터(배치) 학습에서 gradient의 부호만 쓰던 이전 방법 **Rprop**(resilient backpropagation)에서 왔다. RMSProp은 논문이 아니라 Geoffrey Hinton의 2012년 Coursera 강의 *Neural Networks for Machine Learning*에서 Rprop을 미니배치 학습에 맞게 바꾼 방법으로 소개되었다.

```python
class RMSProp:
    """h <- rho * h + (1 - rho) * g*g,   W <- W - lr * g / (sqrt(h) + eps)"""
    def __init__(self, lr=0.01, rho=0.9, eps=1e-7):
        self.lr = lr
        self.rho = rho
        self.eps = eps
        self.h = None

    def update(self, params, grads):
        if self.h is None:
            self.h = {key: np.zeros_like(val) for key, val in params.items()}
        for key in params:
            self.h[key] = self.rho * self.h[key] + (1 - self.rho) * grads[key] * grads[
    key]
            params[key] -= self.lr * grads[key] / (np.sqrt(self.h[key]) + self.eps)
```



같은 학습률 $\alpha = 0.5$로 60단계: AdaGrad는 점점 느려져 멈추고, RMSProp은 최솟값 근처까지 간다.

RMSProp의 경로 끝에서 y가 0 근처에서 위아래로 흔들리는 것도 보인다. gradient를 그 크기로 나누면 gradient가 아주 작아져도 보폭이 대략 α로 유지되기 때문이다. 그래서 실제 학습에서는 학습이 진행될수록 학습률을 줄이기도 한다.

1.7 Adam

아이디어: 모멘텀 + RMSProp. 모멘텀은 **방향**을, RMSProp은 **매개변수별 보폭**을 개선하였다. Adam(Adaptive Moment Estimation)은 두 가지를 합친 방법이다. gradient의 지수이동평균 $\boldsymbol{m}$ (모멘텀의 역할)과 gradient 제곱의 지수이동평균 $\boldsymbol{v}$ (RMSProp의 역할)를 함께 사용한다. 둘 다 W와 같은 shape의 벡터이다. (Adam의 $\boldsymbol{v}$는 모멘텀의 속도 $\boldsymbol{v}$와 다른 양이다.)

갱신 규칙. $k = 1, 2, \ldots$ 번째 갱신에서 ($\boldsymbol{g}_k$는 W_{k-1}에서의 gradient $\frac{\partial L}{\partial W}$, $\boldsymbol{m}_0 = \boldsymbol{v}_0 = \mathbf{0}$)

$$\boldsymbol{m}_k = \beta_1 \boldsymbol{m}_{k-1} + (1-\beta_1)\,\boldsymbol{g}_k, \qquad \boldsymbol{v}_k = \beta_2 \boldsymbol{v}_{k-1} + (1-\beta_2)\,\boldsymbol{g}_k \odot \boldsymbol{g}_k,$$

$$\hat{\boldsymbol{m}}_k = \frac{\boldsymbol{m}_k}{1-\beta_1^k}, \qquad \hat{\boldsymbol{v}}_k = \frac{\boldsymbol{v}_k}{1-\beta_2^k},$$

$$W_k = W_{k-1} - \alpha\,\frac{\hat{\boldsymbol{m}}_k}{\sqrt{\hat{\boldsymbol{v}}_k} + \varepsilon}$$

보통 $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$을 사용한다. AdaGrad, RMSProp과 마찬가지로 제곱, 제곱근, 나눗셈은 성분별이다. 매개변수 w_i 하나에 대해서는 $w_{k,i} = w_{k-1,i} - \alpha\,\hat{m}_{k,i}/(\sqrt{\hat{v}_{k,i}} + \varepsilon)$이다.

- $\boldsymbol{m}$: gradient의 평균 ≈ "어느 방향으로 가야 하는가" (1차 moment)
- $\boldsymbol{v}$: gradient 제곱의 평균 ≈ "이 매개변수의 gradient는 보통 얼마나 큰가" (2차 moment)
- $\hat{\boldsymbol{m}}/\sqrt{\hat{\boldsymbol{v}}}$: 성분마다 평균 방향을 그 크기로 나누어, 매개변수마다 보폭을 비슷한 수준으로 맞춘다.

편향 보정 (bias correction). $\boldsymbol{m}$과 $\boldsymbol{v}$를 $\mathbf{0}$에서 시작하므로 초기에는 실제 평균보다 작게 치우친다. 예를 들어 첫 단계에서 $\boldsymbol{m}_1 = (1-\beta_1)\boldsymbol{g}_1 = 0.1\,\boldsymbol{g}_1$로, 실제 gradient의 $\frac{1}{10}$밖에 되지 않는다. 이를 $1-\beta_1^k$로 나누면

$$\hat{\boldsymbol{m}}_1 = \frac{0.1\,\boldsymbol{g}_1}{1-0.9} = \boldsymbol{g}_1$$

로 바로잡힌다. k가 커지면 $\beta_1^k \to 0$이므로 보정의 효과는 사라진다. $\boldsymbol{v}$도 같은 방식으로 보정한다.

예제: 첫 단계. 편향 보정 덕분에 $\hat{\boldsymbol{m}}_1 = \boldsymbol{g}_1$, $\hat{\boldsymbol{v}}_1 = \boldsymbol{g}_1 \odot \boldsymbol{g}_1$이므로 ($\varepsilon$은 무시)

$$W_1 = W_0 - \alpha \frac{\boldsymbol{g}_1}{|\boldsymbol{g}_1|}$$

이고, AdaGrad처럼 각 좌표가 정확히 α만큼 움직인다. $\alpha = 0.3$이면 $(x_1, y_1) = (-6.7,\ 1.7)$이다.

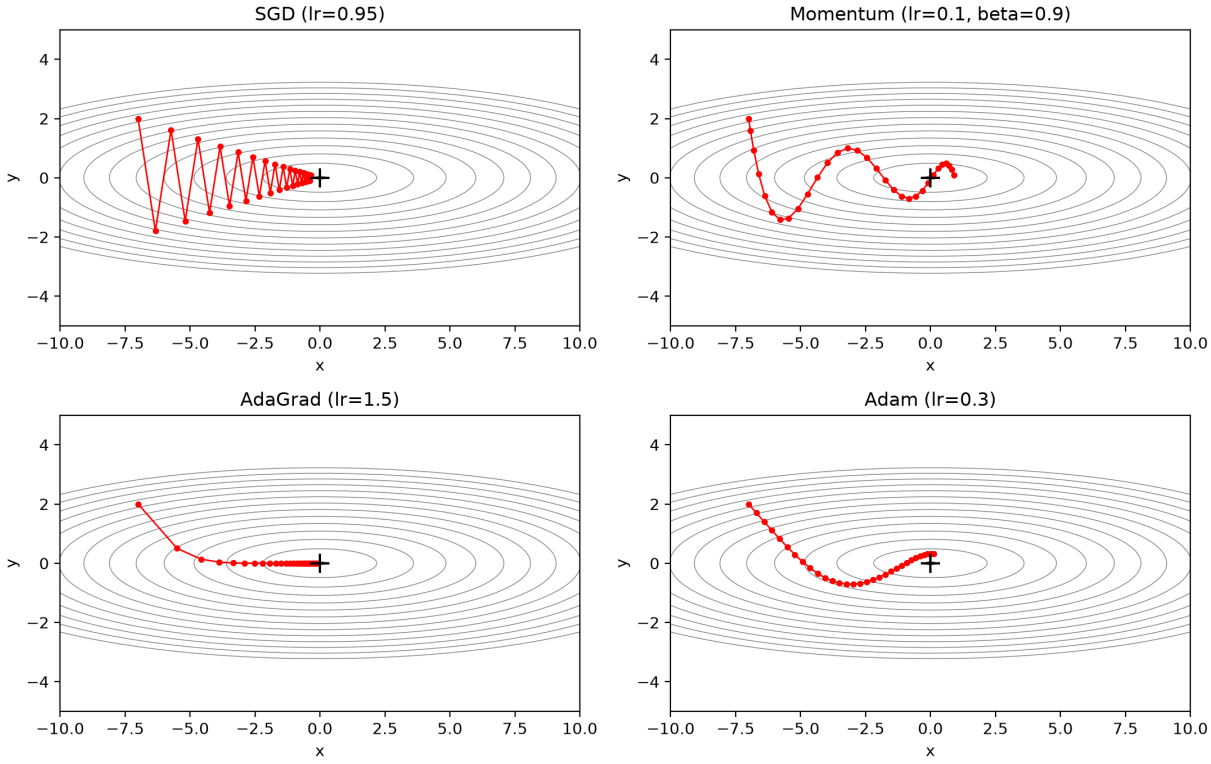
```
class Adam:
    """m <- b1*m + (1-b1)*g,   v <- b2*v + (1-b2)*g*g,
       W <- W - lr * m_hat / (sqrt(v_hat) + eps)   (m_hat, v_hat: bias-corrected)"""
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps
        self.k = 0
        self.m = None
        self.v = None

    def update(self, params, grads):
        if self.m is None:
            self.m = {key: np.zeros_like(val) for key, val in params.items()}
            self.v = {key: np.zeros_like(val) for key, val in params.items()}
        self.k += 1
        for key in params:
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * grads[key]
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) * grads[key] ** 2
            m_hat = self.m[key] / (1 - self.beta1 ** self.k)     # bias correction
            v_hat = self.v[key] / (1 - self.beta2 ** self.k)
            params[key] -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

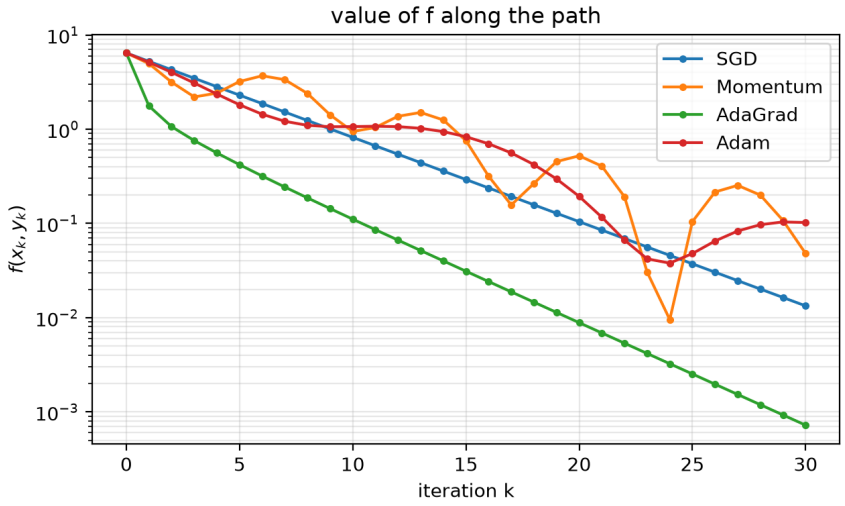
1.8 네 가지 기법의 비교

같은 시작점 $(-7, 2)$에서 30단계씩 갱신한 결과이다. 학습률은 각 기법이 잘 동작하도록 따로 정하였다 (SGD 0.95, 모멘텀 0.1 ($\beta = 0.9$), AdaGrad 1.5, Adam 0.3).

```
optimizers = {
    "SGD (lr=0.95)":                SGD(lr=0.95),
    "Momentum (lr=0.1, beta=0.9)":  Momentum(lr=0.1, beta=0.9),
    "AdaGrad (lr=1.5)":             AdaGrad(lr=1.5),
    "Adam (lr=0.3)":                Adam(lr=0.3),
}
paths = {name: minimize(opt) for name, opt in optimizers.items()}
```



네 가지 최적화 기법의 경로 (+: 최솟값 $(0, 0)$)



경로를 따라 계산한 $f(x_k, y_k)$ (세로축은 로그 눈금)

기법	첫 두 단계 $(x_1, y_1), (x_2, y_2)$	경로의 특징
SGD	$(-6.335, -1.8), (-5.733, 1.62)$	y 방향으로 지그재그, x 방향은 느림
모멘텀	$(-6.93, 1.6), (-6.798, 0.92)$	공이 굴러가듯 크게 휘며 가속, 최솟값을 지나쳤다 돌아옴
AdaGrad	$(-5.5, 0.5), (-4.573, 0.136)$	y 진동을 바로 잡고 골짜기 바닥을 따라 곧게 이동
Adam	$(-6.7, 1.7), (-6.400, 1.402)$	모멘텀처럼 휘지만 진동이 훨씬 작음

매개변수 갱신 기법 정리

기법	갱신 규칙	개선한 점
SGD	$W \leftarrow W - \alpha g$	(기준)
모멘텀	$v \leftarrow \beta v - \alpha g,\ W \leftarrow W + v$	방향: 진동 상쇄, 가속
AdaGrad	$h \leftarrow h + g \odot g,\ W \leftarrow W - \alpha\, g/\sqrt{h}$	보폭: 매개변수별 학습률
RMSProp	$h \leftarrow \rho h + (1-\rho) g \odot g,\ W \leftarrow W - \alpha\, g/\sqrt{h}$	학습률이 0으로 줄지 않음
Adam	m, v의 지수이동평균 + 편향 보정	모멘텀 + RMSProp

어떤 기법이 가장 좋은가?

이 예제에서는 AdaGrad가 가장 빨리 수렴하였지만, 이것으로 순위를 정할 수는 없다. 결과는 함수의 모양, 시작점, 학습률 같은 hyper-parameter에 따라 크게 달라진다. 실제 딥러닝에서는 Adam이 hyper-parameter에 덜 민감하여 기본값으로 많이 쓰이고, 잘 조정한 SGD(+ 모멘텀)가 최종 성능이 더 좋은 경우도 많다. PyTorch에서는 `torch.optim.SGD(..., momentum=0.9)`, `torch.optim.Adagrad`, `torch.optim.RMSprop`, `torch.optim.Adam`으로 제공된다.

1.9 MNIST에서의 비교

2변수 함수에서 본 차이가 실제 신경망 학습에서도 나타날까? 지난주 과제에서 사용한 심층신경망 `MultiLayerNet` (은닉층 4개, 각 100개 node, ReLU, 가중치 초기화 표준편차 0.05)을 네 가지 optimizer로 학습시켜 비교하자. 공정한 비교를 위해 **같은 초기 가중치, 같은 미니배치 순서**를 사용하고, **갱신 규칙 한 줄만** 바꾼다. 학습률은 각 기법에서 흔히 쓰는 값으로 정하였다.

SGD 0.01, 모멘텀 0.01, AdaGrad 0.01, Adam 0.001

```
optimizers = {
    "SGD":      SGD(lr=0.01),
    "Momentum": Momentum(lr=0.01, beta=0.9),
    "AdaGrad":  AdaGrad(lr=0.01),
    "Adam":     Adam(lr=0.001),
}
hidden = [100, 100, 100, 100]      # 4 hidden layers, 100 nodes each
epochs = 5
batch_size = 128

N = x_train.shape[0]
iter_per_epoch = N // batch_size

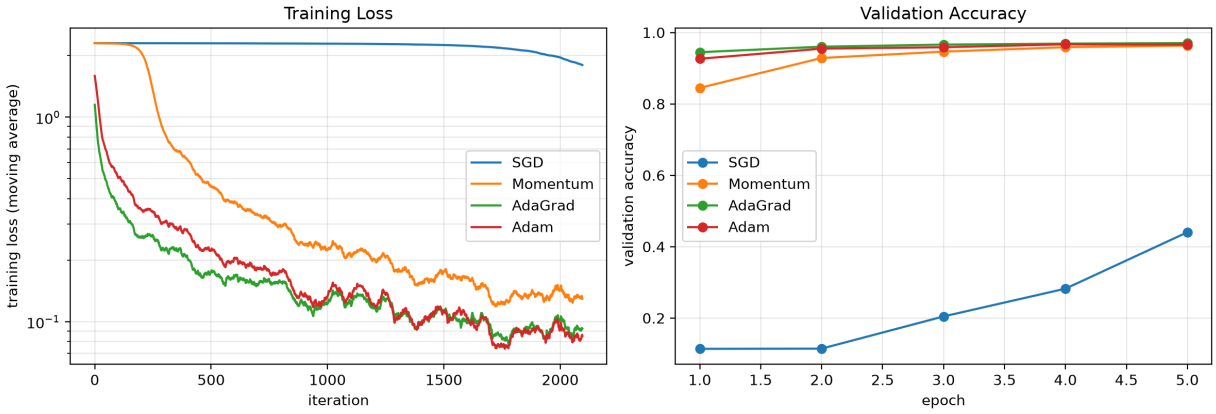
loss_hist = {}
val_hist = {}
for name, optimizer in optimizers.items():
    np.random.seed(1)                                    # the SAME initial weights for
    every optimizer
    net = MultiLayerNet(784, hidden, 10, activation="relu", loss="ce")
```

```
    np.random.seed(2)                                    # the SAME mini-batches for
    every optimizer
    loss_hist[name], val_hist[name] = [], []
    t0 = time.time()
    for epoch in range(epochs):
        perm = np.random.permutation(N)
        for it in range(iter_per_epoch):
            idx = perm[it * batch_size:(it + 1) * batch_size]
            loss = net.loss(x_train[idx], t_train[idx])
            grads = net.backward()
            optimizer.update(net.params, grads)          # <- the only line that differs
            loss_hist[name].append(loss)
        val_hist[name].append(net.accuracy(x_val, t_val))
```

전체 코드는 `mnist_optimizer_compare.py`에 있다. 데이터, network, optimizer는 모듈에서 불러온다.

```
from mnist_data import load_mnist_split
from layers import MultiLayerNet
from optimizers import SGD, Momentum, AdaGrad, Adam
```

`layers.py`의 계층과 `MultiLayerNet`은 지난주 `mnist_mlp.py`와 똑같고, `load_mnist_split()`은 지난주와 같은 훈련/검증 분할을 돌려준다.



iteration에 따른 훈련 손실 (50 iteration 이동평균, 로그 눈금, 왼쪽)과 epoch별 검증 정확도 (오른쪽)

	검증 정확도				
optimizer	1 epoch	2 epoch	3 epoch	4 epoch	5 epoch
SGD (0.01)	11.4%	11.4%	20.5%	28.2%	44.0%
모멘텀 (0.01)	84.5%	92.9%	94.6%	95.9%	96.3%
AdaGrad (0.01)	94.5%	96.0%	96.6%	96.9%	97.0%
Adam (0.001)	92.6%	95.5%	95.9%	96.7%	96.6%

- **SGD**: 처음 1800 iteration 가까이 손실이 $\log 10 \approx 2.30$ 근처에서 거의 줄지 않는다. 손실이 거의 평평한 구간(**plateau**)에 머물러 gradient가 아주 작기 때문이다. 10개 class를 모두 $\frac{1}{10}$의 확률로 찍는 상태가 손실 $\log 10$이다.

- **모멘텀**: 처음에는 SGD처럼 멈춰 있다가, 약 200 iteration 뒤에 plateau를 빠져나온다. 같은 방향의 작은 gradient가 속도에 **누적**되어 보폭이 점점 커졌기 때문이다 (1.5절: 최대 $\frac{1}{1-\beta} = 10$ 배).
- **AdaGrad, Adam**: 처음부터 빠르게 내려간다. gradient를 그 크기로 나누므로, gradient가 아주 작은 plateau에서도 보폭이 대략 α 수준으로 유지된다 (1.6절: 첫 단계에서 각 좌표가 gradient의 크기와 관계없이 α 만큼 움직였던 것).

이 결과를 어떻게 읽어야 할까?

SGD가 느렸던 것은 SGD 자체가 나빠서라기보다, **깊은 network와 작은 초기 가중치** 때문에 처음 gradient가 매우 작은 상황에서 학습률 0.01이 너무 작았기 때문이다. 학습률을 키우거나(연습문제 5), 가중치를 더 잘 초기화하면 SGD도 잘 학습한다. 초기 가중치가 학습에 주는 영향은 다음 절 **가중치의 초깃값**에서 자세히 다룬다. 또 5 epoch 뒤의 검증 정확도는 모멘텀, AdaGrad, Adam이 모두 96 ~ 97%로 비슷하다. optimizer의 차이는 주로 **얼마나 빨리, 얼마나 안정적으로** 학습하는가에서 나타난다.

2 가중치의 초깃값

1.9절에서 SGD가 처음 1800 iteration 가까이 거의 학습하지 못한 것은, 깊은 network의 **작은 초기 가중치** 때문에 gradient가 아주 작았기 때문이었다. 학습은 초기 가중치에서 출발하므로, **어디서 출발하느냐**가 학습의 속도와 성패를 크게 좌우한다. 이 절에서는 초기 가중치를 어떻게 정해야 하는지 살펴본다.

2.1 은닉층의 활성화값 분포 실험

다음 실험으로 초기 가중치의 크기가 network 안의 값들에 어떤 영향을 주는지 보자.

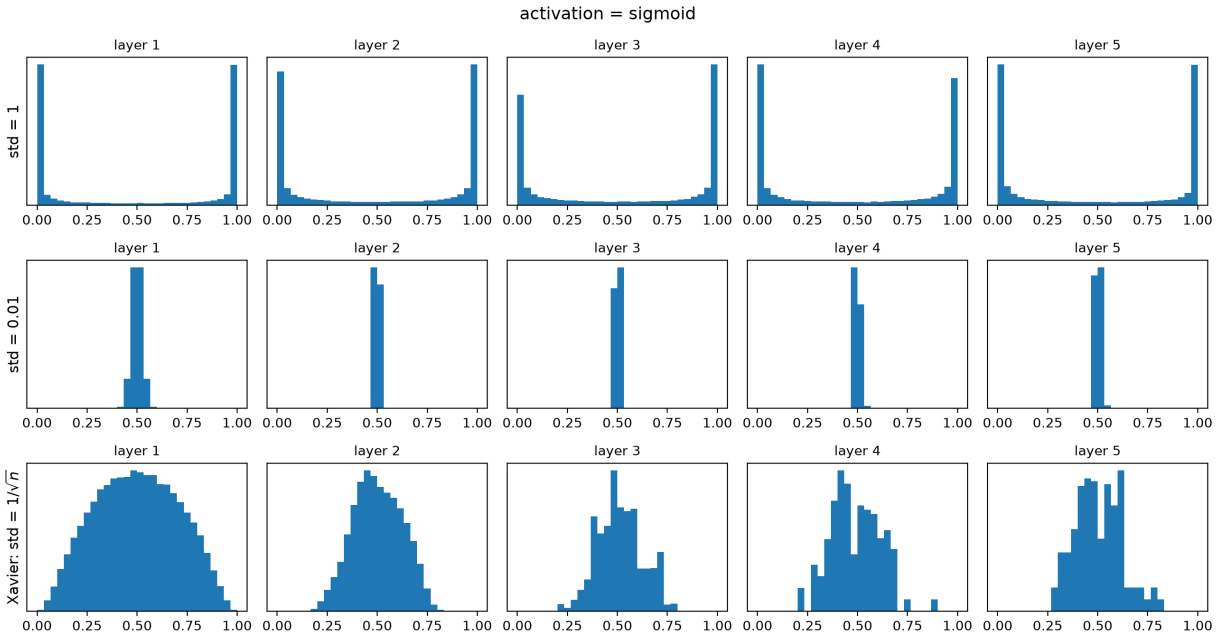
- 입력: 표준정규분포를 따르는 100차원 데이터 1000개 (X: 1000×100)
- network: node가 100개인 층 5개, 편향은 0, **학습은 하지 않고** 순전파만 한다.
- 각 층의 활성화값 Z (활성화 함수를 거친 값)의 **히스토그램**을 그린다.

```
def forward_activations(activation, weight_std, n_layers=5, n_nodes=100, n_data=1000,
    seed=0):
    """Feed 1000 random inputs through n_layers layers; return the list of activations
    Z of each layer."""
    rng = np.random.default_rng(seed)
    X = rng.standard_normal((n_data, n_nodes))          # input data (1000, 100)
    Z = X
    activations = []
    for i in range(n_layers):
        n_in = Z.shape[1]
        std = weight_std(n_in) if callable(weight_std) else weight_std
        W = std * rng.standard_normal((n_in, n_nodes))  # (input nodes, output nodes)
        A = Z @ W                                       # bias = 0
```

```
        Z = activation(A)
        activations.append(Z)
    return activations
```

전체 코드는 `weight_init_activation.py`에 있다.

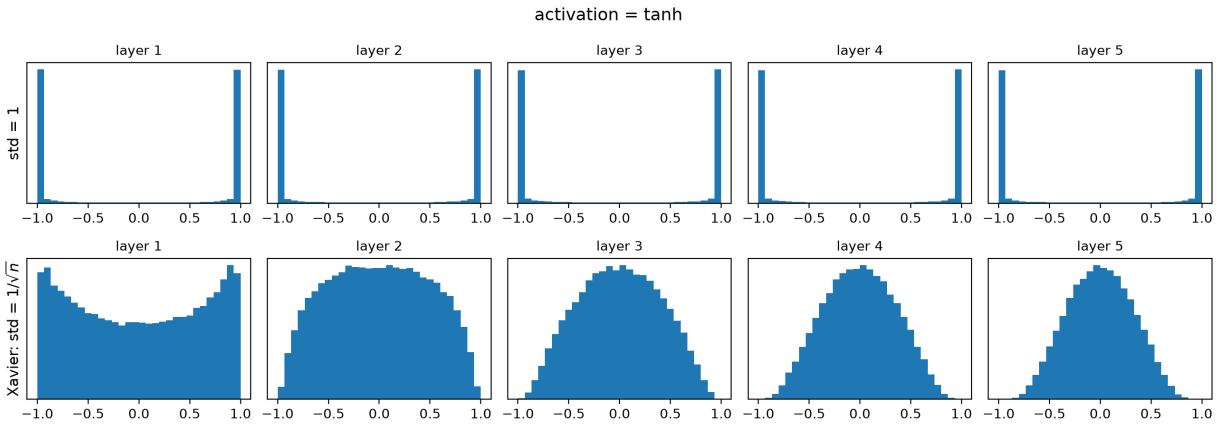
Sigmoid의 경우. 가중치의 표준편차를 1, 0.01, 그리고 뒤에서 설명할 Xavier 초깃값 $\frac{1}{\sqrt{n}}$ (n: 앞 층의 node 수, 여기서는 100이므로 0.1)로 바꾸어 본다.



sigmoid를 쓴 5층 network에서 층별 활성화값의 분포

- **표준편차** 1: 활성화값이 0과 1에 몰려 있다. 가중치가 크면 $A = ZW$의 절댓값이 커져 sigmoid가 양 끝의 평평한 부분에 가기 때문이다. 그곳에서 sigmoid의 미분 $\sigma'(a) = z(1-z)$는 거의 0이다. 역전파에서 gradient는 층을 하나 지날 때마다 $\sigma'(a)$를 곱하므로 (지난주 활성화 계층의 역전파) 앞쪽 층으로 갈수록 gradient가 사라진다. 이를 **기울기 소실(vanishing gradient)**이라 한다.
- **표준편차** 0.01: 활성화값이 모두 0.5 근처에 모여 있다. 기울기 소실은 덜하지만, 100개의 node가 거의 **같은 값**을 출력하므로 node를 많이 두어도 서로 구별되는 특징을 표현하기 어렵다 (**표현력 제한**). 또 가중치가 작으면 역전파에서 W^T를 곱할 때마다 gradient도 작아진다.
- **Xavier** ($\frac{1}{\sqrt{n}}$): 층이 깊어져도 활성화값이 **적당히 퍼진** 분포를 유지한다.

뒤쪽 층에서 분포가 조금 좁아지고 울퉁불퉁해지는 것은 sigmoid의 출력이 0이 아니라 0.5를 중심으로 하기 때문이다. 원점에 대칭인 tanh를 쓰면 같은 Xavier 초깃값에서도 분포가 고르게 유지된다.



tanh를 쓴 경우: 표준편차 1이면 ± 1에 몰리고, Xavier 초깃값이면 고르게 퍼진다.

활성화값 분포로 보는 좋은 초깃값

각 층의 활성화값이 **적당히 퍼져 있어야** 한다.

- 양 끝에 몰리면 $\Rightarrow$ 활성화 함수의 미분이 0에 가까워 **기울기 소실**
- 한 값에 몰리면 $\Rightarrow$ node들이 같은 값을 내어 **표현력 제한**

층마다 값의 크기(분산)가 **일정하게 유지**되도록 가중치의 표준편차를 정하면 된다.

2.2 Xavier 초깃값

층을 지날 때 분산이 일정하게 유지되려면 가중치의 분산이 얼마여야 할까? 앞 층의 node가 n개이고, 다음 층의 한 node의 입력이

$$a_j = \sum_{i=1}^{n} z_i \, w_{ij}$$

라 하자. 다음을 가정한다.

- w_{ij}들은 서로 독립이고 평균이 0, 분산이 $\mathrm{Var}(w)$인 같은 분포를 따른다.
- z_i들도 서로 독립이고 평균이 0, 분산이 $\mathrm{Var}(z)$이며, 가중치와 독립이다.

독립인 확률변수의 합의 분산은 분산의 합이고, 평균이 0인 독립 확률변수의 곱은 $\mathrm{Var}(zw) = \mathbb{E}[z^2]\,\mathbb{E}[w^2] = \mathrm{Var}(z)\,\mathrm{Var}(w)$이므로

$$\mathrm{Var}(a_j) = \sum_{i=1}^{n} \mathrm{Var}(z_i \, w_{ij}) = n \,\mathrm{Var}(w)\,\mathrm{Var}(z)$$

이다. 활성화 함수가 원점 근처에서 거의 항등함수라면 (tanh: $\tanh a \approx a$) 다음 층의 활성화값의 분산도 대략 $n\,\mathrm{Var}(w)\,\mathrm{Var}(z)$이다. 층을 지나도 분산이 변하지 않으려면 $n\,\mathrm{Var}(w) = 1$, 즉

$$\boxed{\mathrm{Var}(w) = \frac{1}{n}, \qquad \text{표준편차 } \frac{1}{\sqrt{n}} \qquad \text{(Xavier 초깃값)}}$$

이다. 앞 층의 node가 많을수록 가중치를 작게 해야 한다.

Glorot와 Bengio의 원래 제안

Xavier 초깃값은 Xavier Glorot와 Yoshua Bengio(2010)가 제안하여 **Glorot 초깃값**이라고도 한다. 같은 논리를 역전파에도 적용하면, gradient가 층을 거꾸로 지날 때 분산이 유지되려면 $\mathrm{Var}(w) = \frac{1}{n_{\text{out}}}$ (다음 층의 node 수)이어야 한다. 원래 논문은 두 조건의 절충으로

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

를 제안하였다. 앞뒤 층의 node 수가 같으면 $\frac{1}{n}$과 같다. 균등분포로 뽑을 때는 $\mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}, \sqrt{\frac{6}{n_{\text{in}}+n_{\text{out}}}}\right)$를 쓴다 (구간 $[-r, r]$의 균등분포의 분산은 $\frac{r^2}{3}$).

2.3 He 초깃값

Xavier 초깃값은 활성화 함수가 원점 근처에서 선형이고 원점에 대칭이라는 가정에서 나왔다. ReLU는 이 가정을 만족하지 않는다. ReLU는 **음수인 입력을 모두 0으로** 만들기 때문이다.

a가 평균 0인 대칭 분포를 따르면 절반은 양수, 절반은 음수이므로, $z = \max(0, a)$에 대해

$$\mathbb{E}[z^2] = \frac{1}{2}\,\mathbb{E}[a^2] = \frac{1}{2}\,\mathrm{Var}(a)$$

이다. ReLU를 지날 때마다 신호의 크기(제곱의 평균)가 **절반**으로 줄어드는 것이다. 다음 층의 입력은 (가중치의 평균이 0이므로)

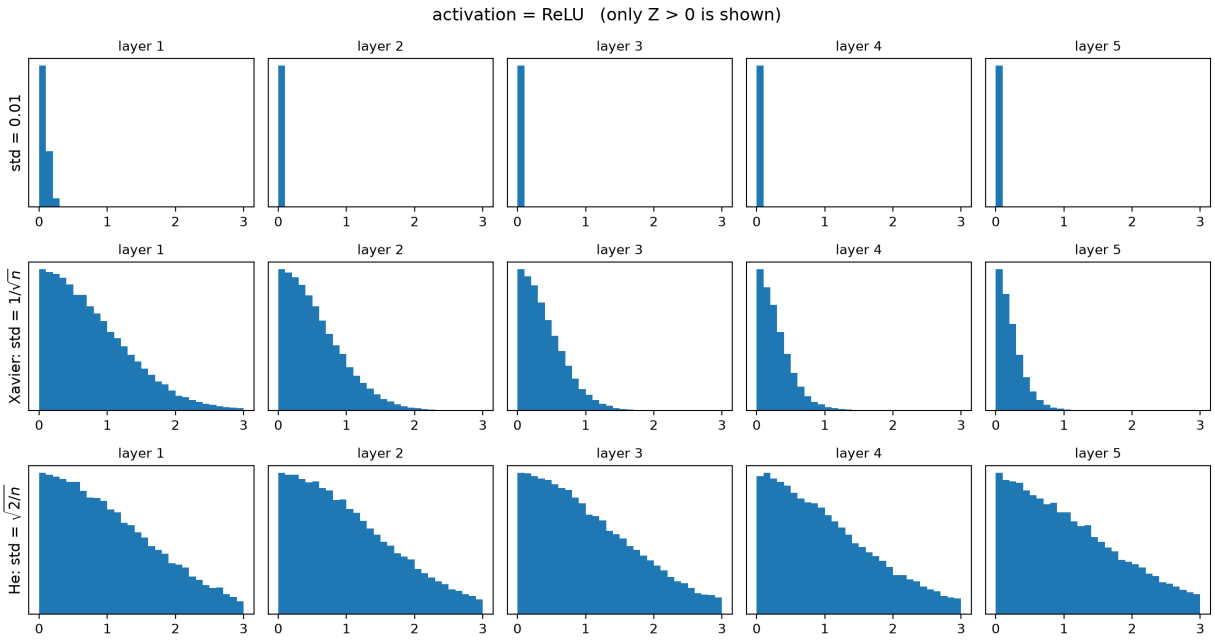
$$\mathrm{Var}\big(a^{(\ell+1)}\big) = n\,\mathrm{Var}(w)\,\mathbb{E}\big[z^2\big] = n\,\mathrm{Var}(w)\cdot\frac{1}{2}\,\mathrm{Var}\big(a^{(\ell)}\big)$$

이다. (z의 평균이 0이 아니므로 $\mathrm{Var}(z)$ 대신 $\mathbb{E}[z^2]$가 들어간다.) 분산이 유지되려면 $\frac{n}{2}\,\mathrm{Var}(w) = 1$, 즉

$$\boxed{\mathrm{Var}(w) = \frac{2}{n}, \qquad \text{표준편차}\ \sqrt{\frac{2}{n}} \qquad \text{(He 초깃값)}}$$

이다. ReLU가 절반을 버리는 만큼 Xavier 초깃값보다 분산을 **두 배**로 키운 것이다 (Kaiming He 등, 2015. **Kaiming 초깃값**이라고도 한다).

ReLU에서의 실험. ReLU를 쓴 5층 network에서 표준편차 0.01, Xavier, He를 비교하면 다음과 같다. (ReLU는 약 절반의 값이 정확히 0이므로, 0이 아닌 값만 그렸다.)



ReLU를 쓴 5층 network에서 층별 활성화값의 분포 ($Z > 0$인 값만)

	층별 활성화값의 표준편차				
초깃값	1층	2층	3층	4층	5층
표준편차 0.01	0.058	0.004	0.0003	0.0000	0.0000
Xavier $\frac{1}{\sqrt{n}}$	0.58	0.41	0.30	0.22	0.18
He $\sqrt{\frac{2}{n}}$	0.82	0.82	0.86	0.88	1.00

- **표준편차** 0.01: 층을 지날 때마다 값이 약 $\frac{1}{14}$로 줄어 4층부터는 사실상 0이다. 순전파의 신호가 사라지면 역전파의 gradient도 사라진다.
- **Xavier**: ReLU가 매번 신호의 절반을 버리므로 층마다 약 $\frac{1}{\sqrt{2}} \approx 0.71$배로 줄어든다. 층이 수십 개가 되면 결국 사라진다.
- **He**: 층이 깊어져도 분포가 **거의 일정하게** 유지된다.

초깃값 정리

활성화 함수	권장 초깃값	가중치의 표준편차
sigmoid, tanh	Xavier (Glorot)	$\sqrt{1/n}$ (또는 $\sqrt{2/(n_{\text{in}} + n_{\text{out}})}$)
ReLU 계열	He (Kaiming)	$\sqrt{2/n}$

n은 앞 층의 node 수 (가중치 행렬 W의 행의 수, 입력 node 수)이다.

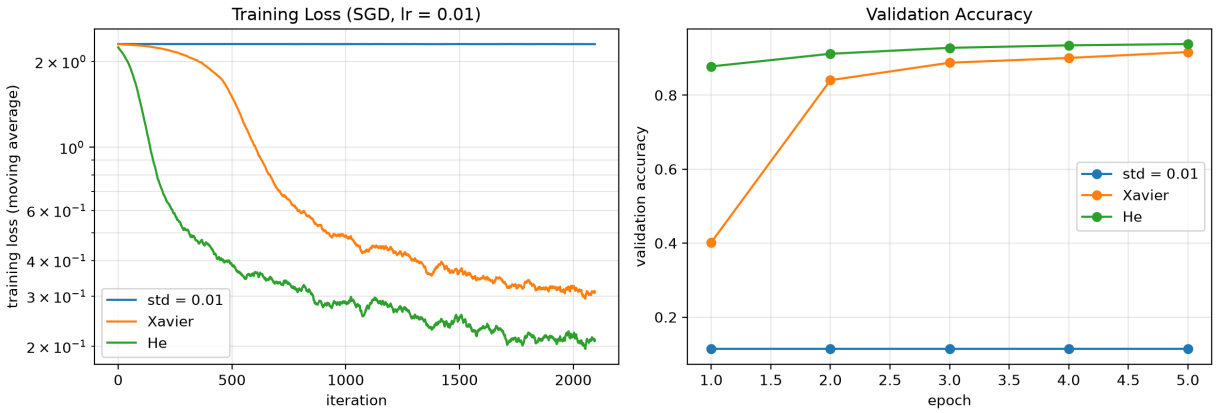
2.4 MNIST에서의 비교

`layers.py`의 `MultiLayerNet`이 `weight_init_std`에 숫자뿐 아니라 `"xavier"`, `"he"`도 받도록 고쳤다.

```
            if weight_init_std == "xavier":             # Var(W) = 1/n_in  (sigmoid, tanh)
                std = np.sqrt(1.0 / n_in)
            elif weight_init_std == "he":               # Var(W) = 2/n_in  (ReLU)
                std = np.sqrt(2.0 / n_in)
            else:                                       # a fixed number, e.g. 0.01
                std = weight_init_std
            W = std * np.random.randn(n_in, n_out)      # (input nodes, output nodes)
```

은닉층 5개 (각 100개 node, ReLU)인 network를 **같은 SGD** (학습률 0.01)로 학습시키고, 초깃값만 바꾸어 비교한다 (`mnist_init_compare.py`).

```
inits = {
    "std = 0.01": 0.01,
    "Xavier":     "xavier",
    "He":         "he",
}
hidden = [100, 100, 100, 100, 100]      # 5 hidden layers, 100 nodes each
```



초깃값에 따른 훈련 손실 (50 iteration 이동평균, 로그 눈금, 왼쪽)과 epoch별 검증 정확도 (오른쪽)

	검증 정확도				
초깃값	1 epoch	2 epoch	3 epoch	4 epoch	5 epoch
표준편차 0.01	11.4%	11.4%	11.4%	11.4%	11.4%
Xavier	40.2%	84.0%	88.8%	90.1%	91.6%
He	87.8%	91.2%	92.8%	93.5%	93.8%

- **표준편차** 0.01: 손실이 $\log 10 \approx 2.30$에서 전혀 줄지 않는다. 앞의 실험에서 본 것처럼 신호가 층을 지나며 사라져 gradient도 거의 0이기 때문이다.
- **Xavier**: 학습은 되지만 처음 500 iteration 정도는 느리다.
- **He**: 처음부터 빠르게 학습한다. ReLU에 맞춘 초깃값이기 때문이다.

optimizer는 똑같이 SGD인데도, 초깃값만 바꾸어 학습이 전혀 되지 않던 network가 잘 학습된다. 1.9절에서 SGD가 느렸던 것도 optimizer보다 **초깃값의 문제**였음을 알 수 있다.

실무에서의 초기화

- Xavier와 He는 지금도 가장 기본이 되는 초기화이다. CNN과 MLP에서는 ReLU면 He, tanh나 sigmoid면 Xavier를 쓰는 것이 표준이다.
- PyTorch의 `nn.Linear`, `nn.Conv2d`는 따로 지정하지 않아도 He(Kaiming) 계열의 균등분포로 초기화된다 (분산이 $\frac{1}{3n}$이 되도록 조정된 값). 직접 지정할 때는 `torch.nn.init.xavier_normal_`, `torch.nn.init.kaiming_normal_` 등을 쓴다.
- Transformer와 대규모 언어모델에서는 작은 고정 표준편차(예: 0.02)의 정규분포를 흔히 쓰고, 층이 깊을수록 잔차(residual) 경로의 출력층을 더 작게 초기화하는 방법을 함께 쓴다.
- 이 밖에 RNN에서의 직교(orthogonal) 초기화, 잔차 블록의 마지막 층을 0으로 초기화하는 방법 등이 있다.
- 다음에 다룰 **배치 정규화**는 각 층의 활성화값 분포를 학습 중에 직접 맞추어 주므로, 초깃값에 대한 민감도를 크게 줄여 준다.

3 배치 정규화

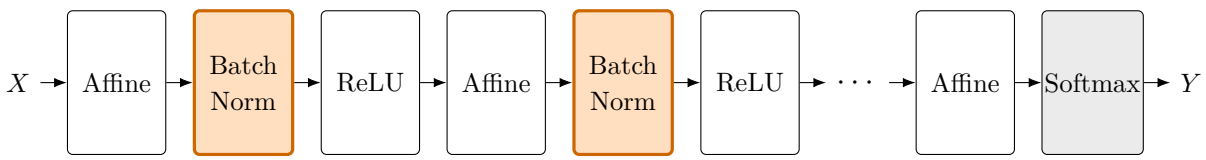
2절에서는 각 층의 활성화값이 적당히 퍼진 분포를 가져야 학습이 잘 된다는 것을 보았고, 그런 분포가 나오도록 **초깃값**을 고르는 방법(Xavier, He)을 배웠다. 그런데 초깃값으로 맞춘 분포는 학습이 시작되어 가중치가 바뀌면 다시 흐트러진다. 그렇다면 아예 **각 층에서 활성화값의 분포를 강제로 맞추어 주면** 어떨까? 이것이 **배치 정규화(Batch Normalization)**의 아이디어이다 (Ioffe & Szegedy, 2015).

배치 정규화의 장점

- 학습을 빨리 진행할 수 있다 (더 큰 학습률을 쓸 수 있다).
- 초깃값에 크게 의존하지 않는다.
- overfitting을 억제한다 (dropout 등의 필요성이 줄어든다).

3.1 배치 정규화 계층을 넣은 신경망

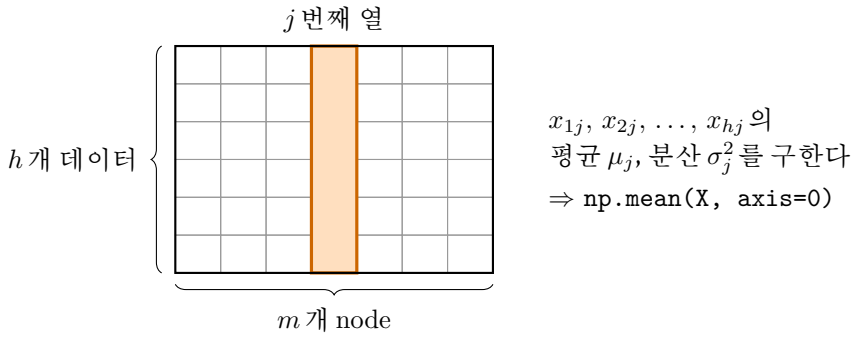
배치 정규화는 **하나의 계층(BatchNorm 계층)**으로 구현하여 신경망에 끼워 넣는다. 보통 Affine 계층과 활성화 함수 계층 사이에 넣는다.



배치 정규화를 사용한 신경망의 예 (주황색이 BatchNorm 계층). 마지막 Affine 계층 뒤에는 넣지 않는다.

3.2 배치 정규화의 순전파

미니배치의 크기를 h, 어떤 층의 node 수를 m이라 하자. Affine 계층의 출력 X는 $h \times m$ 행렬이고, i번째 행은 i번째 데이터, j번째 열은 j번째 node이다. 배치 정규화는 **node(열)마다 따로**, 미니배치에 들어 있는 h개의 데이터에 대하여 평균이 0, 분산이 1이 되도록 정규화한다.



j번째 node에 대하여 (첨자 j는 생략한다)

$$\mu_B = \frac{1}{h}\sum_{i=1}^{h} x_i \qquad \text{(미니배치 평균)} \tag{1}$$

$$\sigma_B^2 = \frac{1}{h}\sum_{i=1}^{h}(x_i - \mu_B)^2 \qquad \text{(미니배치 분산)} \tag{2}$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}} \qquad \text{(정규화)} \tag{3}$$

$$y_i = \gamma\,\hat{x}_i + \beta \qquad \text{(확대 } \gamma \text{와 이동 } \beta\text{)} \tag{4}$$

이다. ε은 0으로 나누는 것을 막는 작은 수 (10^{-7})이다.

- (1)–(3)을 거치면 각 node의 값 $\hat{x}_1, \ldots, \hat{x}_h$는 평균 0, 분산 (거의) 1이 된다.
- 그런데 모든 층의 입력을 평균 0, 분산 1로 고정하면 network의 표현력이 제한될 수 있다. 예를 들어 sigmoid의 입력이 항상 0 근처에만 있으면 sigmoid는 거의 직선처럼 동작한다. 그래서 (4)에서 **학습되는 parameter** γ(scale)와 β(shift)로 분포를 다시 조정할 수 있게 한다. $\gamma = 1$, $\beta = 0$에서 시작하고, W, B와 똑같이 gradient로 갱신한다.
- $\gamma = \sqrt{\sigma_B^2 + \varepsilon}$, $\beta = \mu_B$로 학습되면 원래의 x_i로 되돌아갈 수도 있다. 즉, 정규화가 필요 없으면 network가 스스로 정규화를 "취소"할 수 있다.

노드 사이의 상대적인 크기를 바꾸어도 괜찮을까? 배치 정규화는 node마다 서로 다른 평균과 표준편차를 사용하므로, 하나의 데이터 안에서 node들의 값의 크기와 순서가 바뀔 수 있다. 이는 실제로 신호를 바꾸는 연산이며, 원래의 관계를 모두 보존하는 것은 아니다. 다만 서로 다른 node는 서로 다른 특징을 표현하므로, 출력이 100인 node가 2인 node보다 더 중요하다는 뜻은 아니다. 면적의 단위를 제곱미터에서 제곱센티미터로 바꾸어도 회귀계수를 함께 바꾸면 같은 예측을 할 수 있듯이, 특징의 숫자 크기와 그 특징이 예측에 기여하는 정도는 구분해야 한다. 통계학의 표준화와 수치최적화의 척도 조절에서 얻는 직관은, 특징마다 값의 중심과 크기를 맞추면 경사하강법이 척도

차이에 덜 민감하게 학습할 수 있다는 것이다. 배치 정규화는 이 생각을 학습 중 계속 변하는 내부 특징에 적용하고, 필요한 크기와 중심은 γ, β로 학습하게 한다. 앞뒤 계층도 이 변환을 포함한 전체 network에 맞추어 함께 학습한다. 그러나 배치마다 통계가 달라지므로 고정된 단위 변환과 완전히 같지는 않으며, γ, β가 모든 배치의 원래 값을 복원하는 것도 아니다. 따라서 이는 신호 보존을 보장하는 원리라기보다, 표현을 바꾸는 대가로 학습을 쉽게 하려는 설계이며, 특히 CNN에서 널리 쓰이지만 모든 문제에 유리한 것은 아니다.

행렬로 쓰면, X가 $h \times m$일 때 $\boldsymbol{\mu}_B$, $\boldsymbol{\sigma}_B^2$, $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$는 모두 $1 \times m$이고

$$\hat{X} = \frac{X - \boldsymbol{\mu}_B}{\sqrt{\boldsymbol{\sigma}_B^2 + \varepsilon}}, \qquad Y = \boldsymbol{\gamma} \odot \hat{X} + \boldsymbol{\beta}$$

이다. $1 \times m$ 행벡터가 h개의 행에 broadcast된다는 점은 Affine 계층의 B와 같다.

Affine 계층의 bias B는 필요 없다

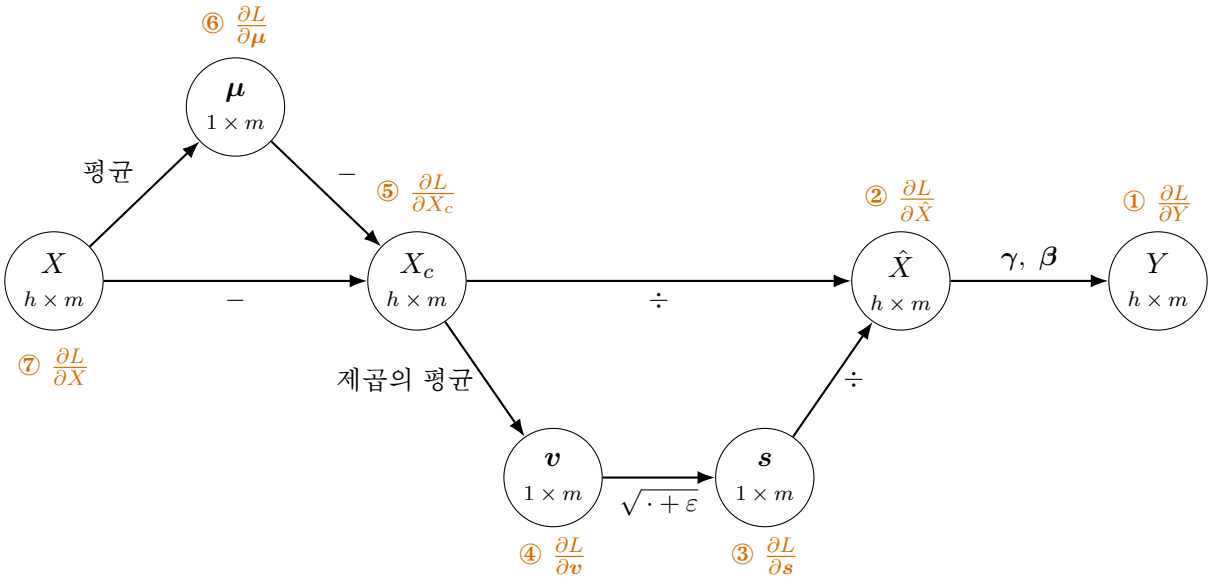
BatchNorm 바로 앞의 Affine 계층의 출력은 $X = X_{\text{in}}W + B$이다. B는 모든 행에 같은 값을 더하므로 평균 $\boldsymbol{\mu}_B$도 정확히 B만큼 커지고, $X - \boldsymbol{\mu}_B$에서 B는 사라진다. 즉 BatchNorm 앞의 B는 출력에 아무 영향이 없고, 그 역할은 $\boldsymbol{\beta}$가 대신한다. 실제로 아래의 gradient 확인에서 $\frac{\partial L}{\partial B_1}$은 0이다. 그래서 PyTorch에서는 보통 `nn.Linear(n_in, n_out, bias=False)` 다음에 `nn.BatchNorm1d(n_out)`를 둔다. 이 강의의 코드에서는 구조를 단순하게 하려고 B를 그대로 두었다.

3.3 배치 정규화의 계산 그래프와 역전파

순전파 (1)–(4)를 작은 계산으로 나누어 변수 노드 기준의 계산 그래프로 그리면 다음과 같다. 중간 변수의 이름은 코드와 같게 하였다.

$$\boldsymbol{\mu} = \frac{1}{h}\sum_i X_{i,:}, \quad X_c = X - \boldsymbol{\mu}, \quad \boldsymbol{v} = \frac{1}{h}\sum_i (X_c)_{i,:}^2, \quad \boldsymbol{s} = \sqrt{\boldsymbol{v} + \varepsilon}, \quad \hat{X} = \frac{X_c}{\boldsymbol{s}}, \quad Y = \boldsymbol{\gamma} \odot \hat{X} + \boldsymbol{\beta}$$

($\boldsymbol{\mu}$: `mu`, X_c: `xc`, $\boldsymbol{v}$: `var`, $\boldsymbol{s}$: `std`, $\hat{X}$: `xn`).



배치 정규화의 계산 그래프 (변수 노드 기준). 검은 화살표가 순전파, 주황색 번호는 역전파에서 gradient를 구하는 순서이다.

역전파는 오른쪽에서 왼쪽으로, ①부터 ⑦까지 차례로 gradient를 구한다. 지금까지의 계산 그래프와 다른 점은 **한 변수에서 화살표가 두 개 이상 나가는 경우**가 있다는 것이다 ($X \to \boldsymbol{\mu}$, $X \to X_c$와 $X_c \to \boldsymbol{v}$, $X_c \to \hat{X}$). 이때 그 변수의 gradient는 **각 경로로 돌아오는 gradient의 합**이다 (다변수 연쇄법칙). 또 $1 \times m$인 변수($\boldsymbol{\mu}, \boldsymbol{v}, \boldsymbol{s}, \boldsymbol{\gamma}, \boldsymbol{\beta}$)는 순전파에서 h개의 행에 broadcast되었으므로, 역전파에서는 Affine 계층의 B와 마찬가지로 **행 방향으로 합한다** (`np.sum(..., axis=0)`).

② $Y = \boldsymbol{\gamma} \odot \hat{X} + \boldsymbol{\beta}$:

$$\frac{\partial L}{\partial \boldsymbol{\beta}} = \sum_i \Big(\frac{\partial L}{\partial Y}\Big)_{i,:}, \qquad \frac{\partial L}{\partial \boldsymbol{\gamma}} = \sum_i \Big(\hat{X} \odot \frac{\partial L}{\partial Y}\Big)_{i,:}, \qquad \frac{\partial L}{\partial \hat{X}} = \boldsymbol{\gamma} \odot \frac{\partial L}{\partial Y}.$$

③ $\hat{X} = X_c / \boldsymbol{s}$에서 $\boldsymbol{s}$로 가는 경로:

$$\frac{\partial L}{\partial \boldsymbol{s}} = -\sum_i \Big(\frac{\partial L}{\partial \hat{X}} \odot \frac{X_c}{\boldsymbol{s}^2}\Big)_{i,:}.$$

④ $\boldsymbol{s} = \sqrt{\boldsymbol{v} + \varepsilon}$:

$$\frac{\partial L}{\partial \boldsymbol{v}} = \frac{\partial L}{\partial \boldsymbol{s}} \cdot \frac{1}{2\sqrt{\boldsymbol{v} + \varepsilon}} = \frac{1}{2\boldsymbol{s}} \frac{\partial L}{\partial \boldsymbol{s}}.$$

⑤ X_c는 $\hat{X}$와 $\boldsymbol{v}$ 두 곳으로 가므로 두 gradient를 더한다. $\boldsymbol{v} = \frac{1}{h}\sum_i (X_c)^2_{i,:}$에서 $\frac{\partial \boldsymbol{v}}{\partial (X_c)_{i,:}} = \frac{2}{h}(X_c)_{i,:}$이므로

$$\frac{\partial L}{\partial X_c} = \underbrace{\frac{1}{\boldsymbol{s}} \odot \frac{\partial L}{\partial \hat{X}}}_{\hat{X} \text{ 경로}} + \underbrace{\frac{2}{h} X_c \odot \frac{\partial L}{\partial \boldsymbol{v}}}_{\boldsymbol{v} \text{ 경로}}.$$

⑥ $X_c = X - \boldsymbol{\mu}$에서 $\boldsymbol{\mu}$ (broadcast되었으므로 합한다):

$$\frac{\partial L}{\partial \boldsymbol{\mu}} = -\sum_i \Big(\frac{\partial L}{\partial X_c}\Big)_{i,:}.$$

⑦ X는 X_c와 $\boldsymbol{\mu}$ 두 곳으로 간다. $\boldsymbol{\mu} = \frac{1}{h}\sum_i X_{i,:}$에서 $\frac{\partial \boldsymbol{\mu}}{\partial X_{i,:}} = \frac{1}{h}$이므로

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial X_c} + \frac{1}{h}\frac{\partial L}{\partial \boldsymbol{\mu}} = \frac{\partial L}{\partial X_c} - \frac{1}{h}\sum_i \Big(\frac{\partial L}{\partial X_c}\Big)_{i,:}.$$

②에서는 $\frac{\partial L}{\partial \hat{X}}$와 함께 parameter의 gradient $\frac{\partial L}{\partial \boldsymbol{\beta}}$, $\frac{\partial L}{\partial \boldsymbol{\gamma}}$도 구한다.

한 줄로 정리한 $\frac{\partial L}{\partial X}$

위의 단계를 모두 대입하면 (node 하나에 대하여, $g_i = \frac{\partial L}{\partial \hat{x}_i}$)

$$\frac{\partial L}{\partial x_i} = \frac{1}{\sqrt{\sigma_B^2 + \varepsilon}} \Big(g_i - \frac{1}{h} \sum_{k=1}^{h} g_k - \hat{x}_i \cdot \frac{1}{h} \sum_{k=1}^{h} g_k \hat{x}_k \Big)$$

이다. g_i에서 **평균 방향 성분**과 $\hat{x}$ **방향 성분**을 빼고 σ_B로 나눈 것이다. 따라서 $\sum_i \frac{\partial L}{\partial x_i} = 0$이다. 입력에 같은 상수를 더해도 정규화된 결과는 그대로이므로, 그 방향으로는 gradient가 없는 것이 당연하다. (BatchNorm 앞의 bias B의 gradient가 0인 이유이기도 하다.)

3.4 학습할 때와 추론할 때

학습할 때는 미니배치의 평균과 분산으로 정규화한다. 그런데 학습이 끝난 network로 **데이터 하나**를 분류할 때는 미니배치 평균을 구할 수 없고, 같은 데이터가 어떤 미니배치에 들어 있느냐에 따라 출력이 달라져서도 안 된다. 그래서 학습하는 동안 미니배치 평균과 분산의 **지수이동평균**을 기억해 두었다가

$$\boldsymbol{\mu}_{\text{run}} \leftarrow 0.9\,\boldsymbol{\mu}_{\text{run}} + 0.1\,\boldsymbol{\mu}_B, \qquad \boldsymbol{\sigma}^2_{\text{run}} \leftarrow 0.9\,\boldsymbol{\sigma}^2_{\text{run}} + 0.1\,\boldsymbol{\sigma}^2_B,$$

추론(검증, 시험)할 때는 이 값으로 정규화한다.

$$\hat{X} = \frac{X - \boldsymbol{\mu}_{\text{run}}}{\sqrt{\boldsymbol{\sigma}^2_{\text{run}} + \varepsilon}}, \qquad Y = \boldsymbol{\gamma} \odot \hat{X} + \boldsymbol{\beta}.$$

이 지수이동평균은 Momentum과 Adam의 m, v와 같은 방식이다 (1.5, 1.7절). 그래서 BatchNorm 계층의 `forward`는 학습 중인지를 알려 주는 `train_flg`를 받는다. PyTorch에서 `model.train()`과 `model.eval()`을 구분해야 하는 가장 큰 이유가 이것이다.

3.5 BatchNorm 계층의 구현

`layers.py`에 `BatchNormalization` 계층을 추가하였다. `backward`의 각 줄이 위의 ①–⑦에 그대로 대응한다.

```
class BatchNormalization:
    """Normalize each node (column) over the mini-batch, then scale and shift:
       x_hat = (x - mu) / sqrt(var + eps),   y = gamma * x_hat + beta"""
    def __init__(self, gamma, beta, momentum=0.9, eps=1e-7):
        self.gamma = gamma          # (1, m)
        self.beta = beta            # (1, m)
        self.momentum = momentum
        self.eps = eps
        self.running_mean = None    # used at test time
        self.running_var = None
        self.dgamma = None
        self.dbeta = None

    def forward(self, X, train_flg=True):
```

```
        if self.running_mean is None:
            self.running_mean = np.zeros((1, X.shape[1]))
            self.running_var = np.ones((1, X.shape[1]))
        if train_flg:                                       # statistics of THIS mini-batch
            mu = np.mean(X, axis=0, keepdims=True)          # (1, m)
            xc = X - mu                                     # (h, m)
            var = np.mean(xc ** 2, axis=0, keepdims=True)   # (1, m)
            std = np.sqrt(var + self.eps)
            xn = xc / std                                   # x_hat (h, m)
            self.h, self.xc, self.std, self.xn = X.shape[0], xc, std, xn
            self.running_mean = self.momentum * self.running_mean + (1 - self.momentum) * mu
            self.running_var = self.momentum * self.running_var + (1 - self.momentum) * var
        else:                                               # test time: running averages
            xn = (X - self.running_mean) / np.sqrt(self.running_var + self.eps)
        return self.gamma * xn + self.beta

    def backward(self, dY):
        self.dbeta = np.sum(dY, axis=0, keepdims=True)              # (1, m)
        self.dgamma = np.sum(self.xn * dY, axis=0, keepdims=True)   # (1, m)
        dxn = self.gamma * dY                                       # (h, m)
        dxc = dxn / self.std
        dstd = -np.sum(dxn * self.xc / (self.std * self.std), axis=0, keepdims=True)
        dvar = 0.5 * dstd / self.std
        dxc += (2.0 / self.h) * self.xc * dvar
        dmu = np.sum(dxc, axis=0, keepdims=True)
        return dxc - dmu / self.h                                   # dL/dX (h, m)
```

단계	식	코드
②	$\frac{\partial L}{\partial \boldsymbol{\beta}}$, $\frac{\partial L}{\partial \boldsymbol{\gamma}}$, $\frac{\partial L}{\partial \hat{X}}$	`dbeta`, `dgamma`, `dxn`
③	$\frac{\partial L}{\partial \boldsymbol{s}}$	`dstd`
④	$\frac{\partial L}{\partial \boldsymbol{v}}$	`dvar`
⑤	$\frac{\partial L}{\partial X_c}$ ($\hat{X}$ 경로 + $\boldsymbol{v}$ 경로)	`dxc = dxn / std, dxc += ...`
⑥	$\frac{\partial L}{\partial \boldsymbol{\mu}}$ (부호를 뺀 값)	`dmu = np.sum(dxc, ...)`
⑦	$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial X_c} - \frac{1}{h}\sum_i(\cdot)$	`return dxc - dmu / self.h`

코드의 `dmu`는 $\sum_i \left(\frac{\partial L}{\partial X_c}\right)_{i,:}$, 즉 $-\frac{\partial L}{\partial \boldsymbol{\mu}}$ 이다.

`MultiLayerNet`에는 `use_batchnorm` 옵션을 추가하였다. `True`이면 은닉층마다 Affine → BatchNorm → 활성화 함수 순서로 계층을 쌓고, $\boldsymbol{\gamma}, \boldsymbol{\beta}$를 `params`에 `gamma1`, `beta1`, ...의 이름으로 넣는다. `params`에 들어 있으므로 optimizer는 W, B와 똑같이 $\boldsymbol{\gamma}, \boldsymbol{\beta}$도 갱신한다.

```
            if i < len(sizes) - 2:                      # hidden layer
                if use_batchnorm:                       # Affine -> BatchNorm -> activation
                    gamma, beta = np.ones((1, n_out)), np.zeros((1, n_out))
                    self.params[f"gamma{i + 1}"] = gamma
                    self.params[f"beta{i + 1}"] = beta
```

```
                    self.layers.append(BatchNormalization(gamma, beta))
                self.layers.append(Relu() if activation == "relu" else Sigmoid())
```

순전파는 `train_flg`를 BatchNorm 계층 (그리고 4.4절의 Dropout 계층)에만 넘긴다. 손실(학습)에서는 미니배치 통계를, 정확도(평가)에서는 이동평균을 쓴다.

```
    def predict(self, X, train_flg=False):    # scores U
        for layer in self.layers:
            if isinstance(layer, (BatchNormalization, Dropout)):   # training and test differ
                X = layer.forward(X, train_flg)
            else:
                X = layer.forward(X)
        return X

    def loss(self, X, T):                     # used for training: batch statistics, dropout
        data_loss = self.last.forward(self.predict(X, train_flg=True), T)
```

(`loss`의 나머지 부분인 가중치 감소는 4.2절에서 다룬다.) 역전파가 끝나면 각 계층에 저장된 gradient를 모은다. BatchNorm 계층의 번호는 바로 앞 Affine 계층의 번호를 따른다.

```
        grads, k = {}, 0
        for layer in self.layers:
            if isinstance(layer, Affine):
                k += 1
                grads[f"W{k}"], grads[f"B{k}"] = layer.dW, layer.dB
            elif isinstance(layer, BatchNormalization):
                grads[f"gamma{k}"], grads[f"beta{k}"] = layer.dgamma, layer.dbeta
```

gradient 확인. 계산 그래프로 유도한 역전파가 맞는지, 작은 network에서 수치미분과 비교한다 (`mnist_batchnorm.py`의 (1)).

```
np.random.seed(0)
small = MultiLayerNet(4, [5, 5], 3, weight_init_std=1.0, use_batchnorm=True)
X_small = np.random.randn(6, 4)
T_small = np.eye(3)[np.random.randint(0, 3, size=6)]
small.loss(X_small, T_small)
grads = small.backward()
for key in small.params:
    num = numerical_gradient(lambda: small.loss(X_small, T_small), small.params[key])
    print(f"{key:7s}: max |backprop - numerical| = {np.max(np.abs(grads[key] - num)):.2e}")
```

```
W1     : max |backprop - numerical| = 2.48e-11
B1     : max |backprop - numerical| = 2.43e-17
gamma1 : max |backprop - numerical| = 2.48e-11
beta1  : max |backprop - numerical| = 1.64e-11
W2     : max |backprop - numerical| = 2.35e-11
B2     : max |backprop - numerical| = 1.11e-11
gamma2 : max |backprop - numerical| = 1.31e-11
```

```
beta2  : max |backprop - numerical| = 1.31e-11
W3     : max |backprop - numerical| = 2.85e-11
B3     : max |backprop - numerical| = 1.41e-11
```

모든 parameter에서 오차가 10^{-11} 이하이므로 역전파가 올바르다. B_1, B_2는 BatchNorm 바로 앞의 bias이므로 역전파로 구한 gradient가 정확히 0이다. (B_1의 오차가 특히 작은 것은 수치미분도 정확히 0이 나왔기 때문이다.)

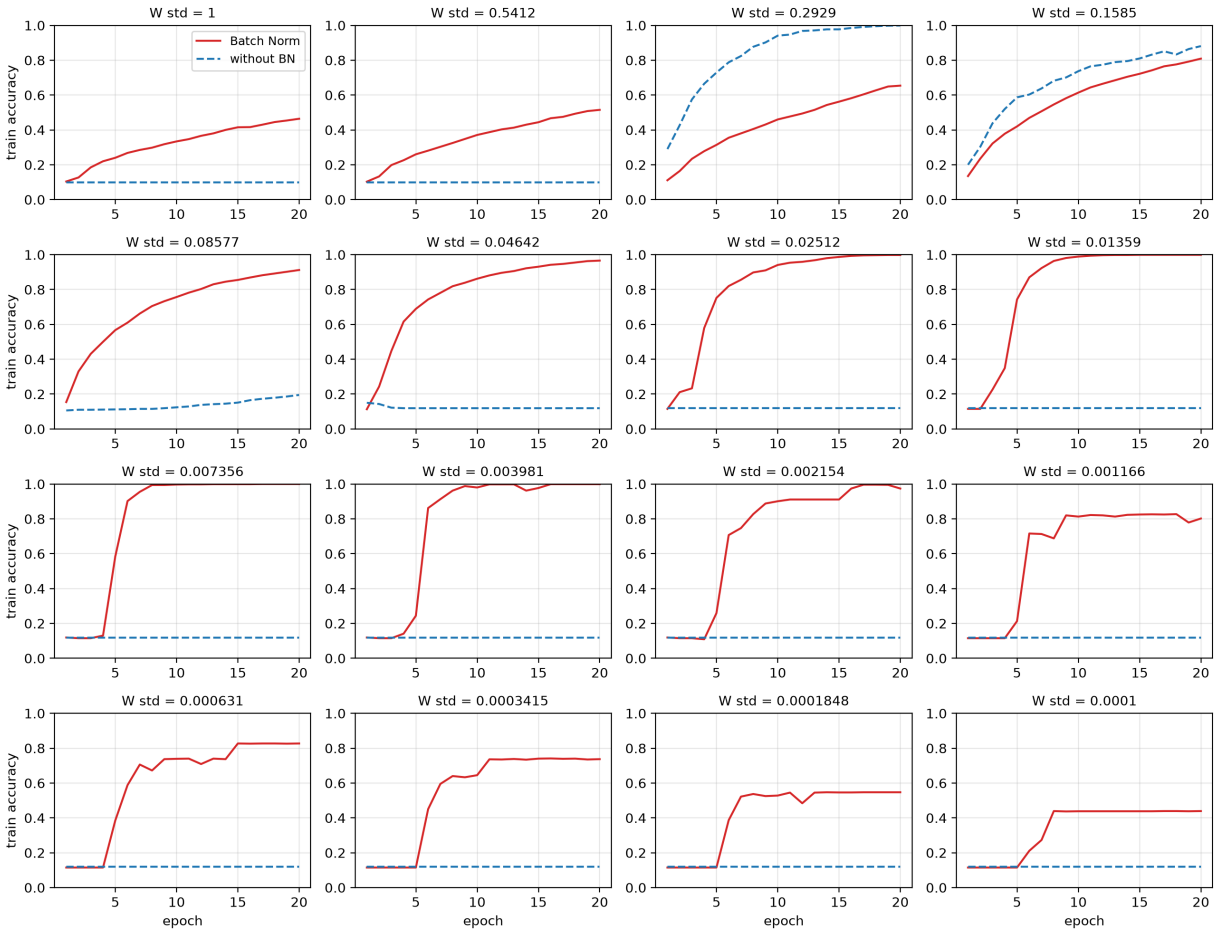
3.6 배치 정규화의 효과

실험 1: 가중치의 초깃값을 바꾸어 가며 (책의 실험). MNIST의 훈련 데이터 1000장만으로, 은닉층 5개 (각 100개 node, ReLU)인 network를 SGD (학습률 0.01, 미니배치 100)로 20 epoch 학습시킨다. 가중치의 표준편차를 1부터 0.0001까지 16가지로 바꾸어 가며, **같은 초깃값, 같은 미니배치**로 배치 정규화가 있을 때와 없을 때를 비교한다.

```
x_small, t_small = x_train[:1000], t_train[:1000]
hidden = [100, 100, 100, 100, 100]               # 5 hidden layers, 100 nodes each
weight_scales = np.logspace(0, -4, num=16)       # 1, ..., 0.0001
epochs_small = 20

fig, axes = plt.subplots(4, 4, figsize=(13, 10))
t0 = time.time()
for ax, w in zip(axes.ravel(), weight_scales):
    curves = {}
    for use_bn in (True, False):
        np.random.seed(1)                         # the SAME initial weights
        net = MultiLayerNet(784, hidden, 10, weight_init_std=w, use_batchnorm=use_bn)
        _, accs = train(net, x_small, t_small, epochs_small, 100, 0.01, x_small,
    t_small)
        curves[use_bn] = accs
```

`train`은 SGD로 학습하며 epoch마다 정확도를 기록하는 함수이다 (코드 파일 참고).



가중치 초깃값의 표준편차별 훈련 정확도. 실선: 배치 정규화 사용, 점선: 사용하지 않음

가중치 표준편차	1	0.29	0.16	0.086	0.025	0.0074	0.0012	0.0001
배치 정규화 사용	0.46	0.65	0.81	0.91	**1.00**	**1.00**	0.80	0.44
사용하지 않음	0.10	**1.00**	0.88	0.20	0.12	0.12	0.12	0.12

20 epoch 후의 훈련 정확도 (16가지 중 일부)

- **배치 정규화가 없으면** 표준편차가 0.29, 0.16 근처일 때만 학습이 되고, 그보다 작으면 (2절에서 본 것처럼 신호가 사라져) 정확도가 0.12 근처에 머문다. 표준편차가 1이면 활성화값이 폭발하여 역시 학습이 되지 않는다. 학습이 되는 초깃값의 범위가 매우 좁다.
- **배치 정규화를 쓰면** 거의 모든 초깃값에서 학습이 진행되고, 표준편차 0.05 ~ 0.002에서는 20 epoch 안에 훈련 정확도 96 ~ 100%에 도달한다. 초깃값에 대한 의존성이 크게 줄었다.
- 표준편차 0.29, 0.16처럼 원래 잘 되는 초깃값에서는 배치 정규화가 오히려 느릴 수도 있다. 배치 정규화가 항상 모든 경우에 더 좋은 것은 아니다.

왜 초깃값이 너무 크거나 작으면 배치 정규화를 써도 느릴까?

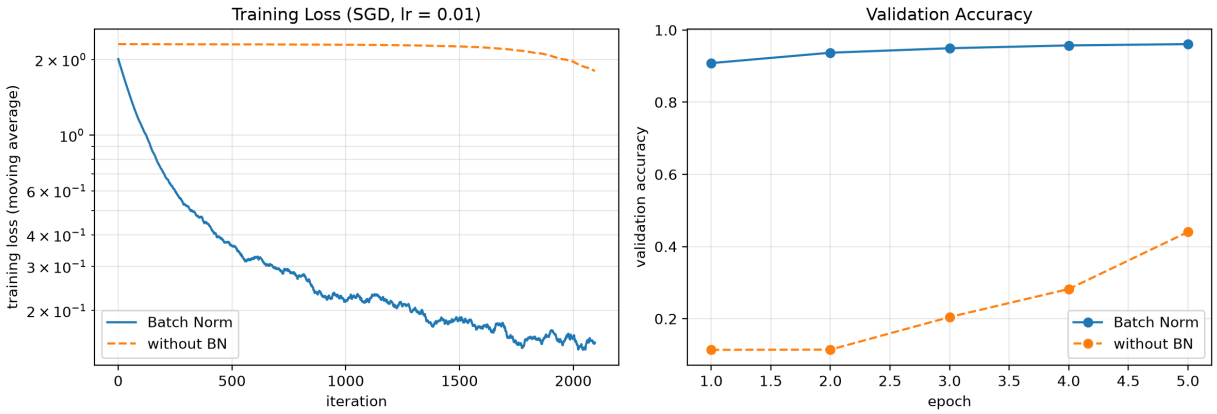
BatchNorm 앞의 가중치를 $W \to aW$로 a배 하면 X와 $\boldsymbol{\mu}_B, \boldsymbol{\sigma}_B$도 모두 a배가 되므로 정규화된 $\hat{X}$는 **변하지 않는다**. 즉 손실은 W의 크기와 무관하다. 반면 $\frac{\partial L}{\partial (aW)} = \frac{1}{a}\frac{\partial L}{\partial W}$이므로 gradient는 $\frac{1}{a}$배가 되고, 갱신량을 W의 크기에 대한 비율로 보면 $\frac{1}{a^2}$배가 된다.

- 가중치가 크면 ($a \gg 1$) 상대적인 갱신량이 작아서 학습이 **느리다** (표준편차 1, 0.54).
- 가중치가 아주 작으면 ($a \ll 1$) 상대적인 갱신량이 매우 커서 학습이 **불안정**하다. 처음 몇 epoch 동안 정체하다가 갑자기 뛰어오르고, 정확도가 중간에서 멈추기도 한다 (표준편차 0.001 이하).

배치 정규화를 쓰면 가중치의 크기가 사실상 **학습률을 조절하는 역할**을 한다. 그래도 배치 정규화가 없을 때처럼 학습이 완전히 멈추는 경우는 없다.

실험 2: 1.9절의 network에 배치 정규화를 넣으면. 1.9절에서 SGD (학습률 0.01)는 은닉층 4개 (각 100개 node, 표준편차 0.05)인 network를 5 epoch 동안 거의 학습시키지 못했다 (검증 정확도 44.0%). 같은 network, 같은 초깃값, 같은 SGD에 배치 정규화만 넣어 전체 MNIST로 학습시킨다 (`mnist_batchnorm.py`의 (3)).

```
results = {}
for use_bn in (False, True):
    np.random.seed(1)
    net = MultiLayerNet(784, [100, 100, 100, 100], 10, weight_init_std=0.05,
    use_batchnorm=use_bn)
    results[use_bn] = train(net, x_train, t_train, 5, 128, 0.01, x_val, t_val)
```



배치 정규화 유무에 따른 훈련 손실 (50 iteration 이동평균, 로그 눈금, 왼쪽)과 epoch별 검증 정확도 (오른쪽)

	검증 정확도				
	1 epoch	2 epoch	3 epoch	4 epoch	5 epoch
SGD	11.4%	11.4%	20.5%	28.2%	44.0%
SGD + 배치 정규화	90.8%	93.7%	95.0%	95.7%	96.1%

- 배치 정규화가 없으면 손실이 $\log 10 \approx 2.30$ 근처의 plateau에서 1500 iteration 넘게 벗어나지 못한다.
- 배치 정규화를 넣으면 **첫 epoch부터** 검증 정확도가 90%를 넘고, 5 epoch 후에는 96.1%로 1.9절의 Adam (96.6%)에 가깝다. optimizer는 가장 단순한 SGD 그대로이다.
- 검증 정확도는 `train_flg=False`, 즉 학습 중에 모은 **이동평균**으로 계산한 것이다. 이동평균으로도 정확도가 높다는 것은 3.4절의 추론 방식이 잘 동작한다는 뜻이다.

세 가지 해결책의 비교: 같은 문제 / 다른 접근

1.9절에서 SGD가 느렸던 network를 세 가지 방법으로 고쳐 보았다.

- **optimizer를 바꾼다** (Momentum, AdaGrad, Adam): 작은 gradient도 크게 키워 plateau를 빠져나온다.
- **초깃값을 바꾼다** (Xavier, He): 처음부터 신호가 사라지지 않게 한다.
- **배치 정규화를 넣는다**: 학습하는 동안 **계속** 각 층의 분포를 맞춘다.

실제로는 세 가지를 **함께** 쓴다 (예: He 초깃값 + 배치 정규화 + Adam 또는 Momentum SGD).

배치 정규화는 왜 효과가 있을까? 그리고 그 후

- 원래 논문은 앞 층의 가중치가 바뀌면 뒤 층 입력의 분포가 계속 바뀌는 현상 (**internal covariate shift**)을 줄이기 때문이라고 설명하였다. 이후 연구 (Santurkar 등, 2018)에서는 손실 함수의 지형을 **더 매끄럽게** 만들어 큰 학습률을 쓸 수 있게 하는 효과가 더 중요하다고 보고 있다. 아직 완전히 합의된 설명은 없다.
- 미니배치마다 평균과 분산이 조금씩 달라지므로 같은 데이터도 매번 약간 다르게 정규화된다. 이 잡음이 **규제(regularization)** 효과를 낸다.
- 미니배치가 매우 작으면 ($h = 1, 2$) 평균과 분산을 믿을 수 없어 성능이 나빠진다. 그래서 데이터 하나 안에서 정규화하는 **Layer Normalization** (Transformer, 대규모 언어모델의 표준), 채널을 묶어 정규화하는 **Group Normalization** 등이 나왔다. CNN에서는 지금도 배치 정규화가 기본이다.
- PyTorch에서는 `nn.BatchNorm1d` (MLP), `nn.BatchNorm2d` (CNN), `nn.LayerNorm`을 쓴다. PyTorch의 `momentum=0.1`은 이 노트의 0.9와 같은 뜻이다 (새 값의 비율).

4 바른 학습을 위해: 오버피팅, 가중치 감소, 드롭아웃

지금까지는 **훈련 데이터의 손실을 빠르고 안정적으로 줄이는 방법**을 다루었다. 그러나 우리가 정말 원하는 것은 훈련 데이터가 아니라 **처음 보는 데이터**를 잘 맞히는 network이다. 이 절에서는 훈련 데이터에만 지나치게 맞추어지는 **오버피팅(overfitting, 과대적합)**과, 이를 억제하는 대표적인 두 방법인 **가중치 감소(weight decay)**와 **드롭아웃(dropout)**을 다룬다. 이렇게 오버피팅을 억제하는 기법을 통틀어 **규제(regularization)**라고 한다.

4.1 오버피팅

오버피팅은 network가 훈련 데이터에만 지나치게 적응하여, 훈련 데이터에 없는 데이터에는 제대로 대응하지 못하는 상태를 말한다. 주로 다음 두 경우에 일어난다.

- parameter가 많고 표현력이 높은 network
- 훈련 데이터가 적은 경우

실험: 일부러 오버피팅을 일으켜 보자. 책에서와 같이 MNIST 훈련 데이터 중 300**장만** 사용하고, 은닉층 6개 (각 100개 node, ReLU, He 초깃값)인 7층 network를 SGD (학습률 0.01, 미니배치 100)로 300 epoch 학습시킨다. parameter는 약 13만 개인데 데이터는 300개뿐이다. 매 epoch마다 훈련 데이터 300장과 시험 데이터 10000장의 정확도를 기록한다 (`mnist_overfit.py`).

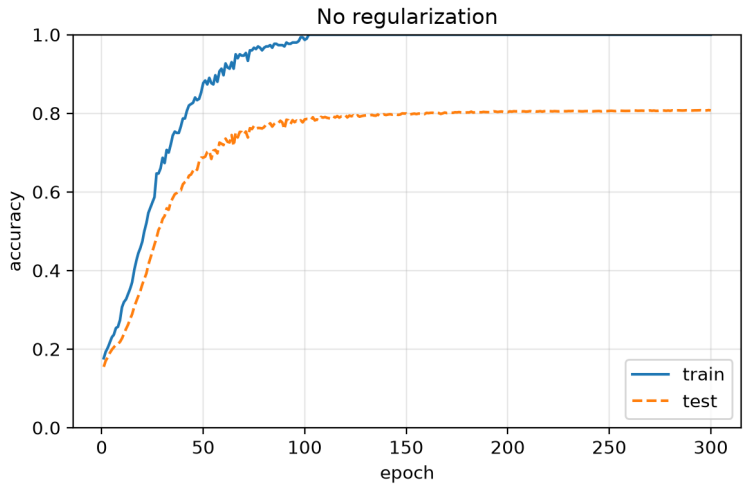
```
x_train, t_train, x_val, t_val, x_test, t_test = load_mnist_split()
x_train, t_train = x_train[:300], t_train[:300]    # only 300 training images ->
    overfitting

hidden = [100, 100, 100, 100, 100, 100]            # 6 hidden layers (7 Affine layers)
epochs = 300
batch_size = 100
```

같은 초깃값, 같은 미니배치로 비교하기 위해 network는 항상 같은 seed로 만든다.

```
def make_net(**options):
    np.random.seed(1)                              # the SAME initial weights
    return MultiLayerNet(784, hidden, 10, weight_init_std="he", **options)
```

```
# (1) overfitting
acc = train(make_net(), SGD(lr=0.01))
```



규제가 없을 때 훈련 데이터(실선)와 시험 데이터(점선)의 정확도

약 100 epoch 만에 훈련 정확도는 100%가 되지만, 시험 정확도는 80.7%에서 더 오르지 않는다. 이 network는 300장의 훈련 데이터를 사실상 **외워 버린** 것이다. 훈련 정확도가 100%가 된 뒤에는 훈련

데이터에서 더 배울 것이 없으므로 시험 정확도도 그대로이다. 훈련 정확도와 시험 정확도의 차이(19.3%p)가 오버피팅의 정도를 나타낸다.

시험 데이터의 역할

여기서 시험 데이터는 학습에는 전혀 쓰지 않고, 오버피팅 현상을 보여 주는 데만 썼다. 아래에서 규제의 세기 λ처럼 사람이 정해야 하는 값을 **고를** 때는 시험 데이터가 아니라 훈련 데이터에서 떼어 둔 **검증 데이터**를 쓴다. 그 이유는 5.1절에서 자세히 다룬다.

4.2 가중치 감소

오버피팅은 가중치 매개변수의 값이 커서 발생하는 경우가 많다. 가중치가 크면 입력이 조금만 달라져도 출력이 크게 바뀌는, 즉 훈련 데이터 하나하나에 날카롭게 맞춘 복잡한 함수가 되기 쉽다. **가중치 감소(weight decay)**는 큰 가중치에 벌칙(penalty)을 주어 이를 억제한다. 손실 함수에 모든 가중치의 제곱합(**L2 norm**의 제곱)을 더한다.

$$L_{\text{total}} = L + \frac{\lambda}{2}\sum_k \|W_k\|^2, \qquad \|W\|^2 = \sum_{i,j} w_{ij}^2$$

λ는 규제의 세기를 정하는 hyperparameter이다. λ가 클수록 큰 가중치에 큰 벌칙을 준다. $\frac{1}{2}$은 미분을 깔끔하게 하려고 붙인 것이다.

벌칙 항의 미분은 $\frac{\partial}{\partial W}\frac{\lambda}{2}\|W\|^2 = \lambda W$이므로, 역전파로 구한 gradient에 λW만 더하면 된다.

$$\frac{\partial L_{\text{total}}}{\partial W} = \frac{\partial L}{\partial W} + \lambda W$$

이것을 SGD 갱신식에 넣으면

$$W \leftarrow W - \alpha\left(\frac{\partial L}{\partial W} + \lambda W\right) = (1-\alpha\lambda)\,W - \alpha\frac{\partial L}{\partial W}$$

이다. 매 단계마다 가중치를 $(1-\alpha\lambda)$배로 **줄인(decay)** 뒤 보통의 SGD 갱신을 하는 것과 같다. 그래서 "가중치 감소"라고 부른다.

무엇에 가중치 감소를 적용하는가

가중치 W에만 적용하고, bias B와 배치 정규화의 $\boldsymbol{\gamma}, \boldsymbol{\beta}$에는 적용하지 않는 것이 보통이다. bias는 node마다 하나뿐이라 parameter 수가 적고 오버피팅에 거의 기여하지 않으며, $\boldsymbol{\gamma}$를 0 쪽으로 끌어당기면 오히려 정규화된 신호를 죽이게 되기 때문이다. 코드에서는 이름이 `"W"`로 시작하는 parameter에만 적용한다.

`MultiLayerNet`에 `weight_decay_lambda` 옵션을 추가하였다(기본값 0). 손실에 벌칙 항을 더하고,

```
    def loss(self, X, T):                    # used for training: batch statistics, dropout
        data_loss = self.last.forward(self.predict(X, train_flg=True), T)
        # weight decay: + (lambda/2) * sum of W^2 over all weight matrices (not B, gamma, beta)
        sum_W2 = sum(np.sum(W ** 2) for key, W in self.params.items() if key.startswith("W"))
        return data_loss + 0.5 * self.weight_decay_lambda * sum_W2
```

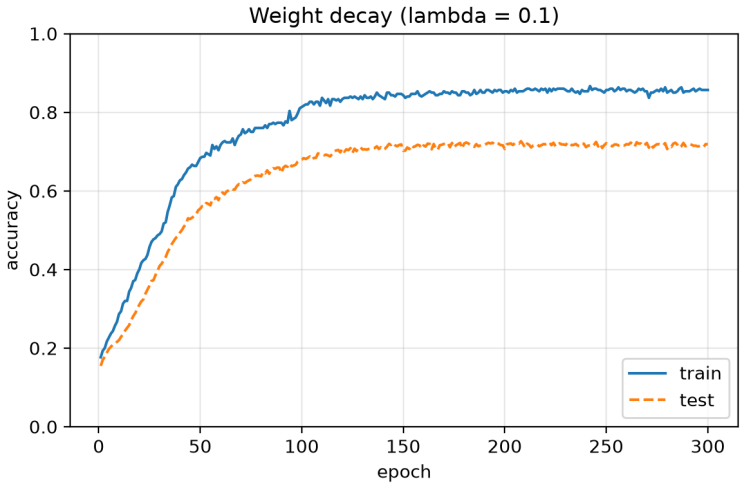
역전파가 끝난 뒤 가중치의 gradient에 λW를 더한다.

```
        for key in grads:                          # weight decay: gradient + lambda * W
            if key.startswith("W"):
                grads[key] = grads[key] + self.weight_decay_lambda * self.params[key]
        return grads
```

수치미분으로 확인하면 ($\lambda = 0.3$) 오차는 10^{-10} 정도로, 벌칙 항의 gradient가 올바르게 구현되었다.

실험. 앞의 실험과 똑같은 조건에서 $\lambda = 0.1$로 가중치 감소를 적용한다.

```
# (2) weight decay
acc = train(make_net(weight_decay_lambda=0.1), SGD(lr=0.01))
```



가중치 감소 ($\lambda = 0.1$)를 적용했을 때의 정확도

훈련 정확도가 100%에 이르지 않고 (85.7%), 훈련과 시험 정확도의 차이가 19.3%p에서 13.9%p로 줄었다. 오버피팅이 억제된 것이다. 다만 이 실험에서는 시험 정확도 자체도 80.7%에서 71.8%로 내려갔다. $\lambda = 0.1$은 이 network에는 **너무 강한** 규제인 것이다.

λ는 얼마로 해야 할까? λ를 바꾸어 가며 학습시키고, **검증 데이터**의 정확도를 비교한다.

```
for lam in (0.0, 0.01, 0.03, 0.1, 0.3):
    net = make_net(weight_decay_lambda=lam)
    train_acc, _ = train(net, SGD(lr=0.01))
    print(f"lambda = {lam:<5}: train {train_acc[-1]:.4f}  validation {net.accuracy(
    x_val, t_val):.4f}")
```

λ	0	0.01	0.03	0.1	0.3
훈련 정확도	100%	100%	100%	85.7%	12.0%
검증 정확도	80.3%	80.6%	**80.9**%	70.8%	11.4%

- λ가 너무 크면 (0.3) 가중치를 키우지 못하여 훈련 데이터조차 학습하지 못한다. 오버피팅의 반대인 **과소적합(underfitting)**이다.
- 적당한 λ (0.03)에서는 검증 정확도가 조금 오른다. 규제의 세기에는 이처럼 **적당한 값**이 있고, 그 값은 검증 데이터로 찾는다.
- 데이터가 300장뿐이므로 어떤 규제를 써도 큰 향상은 어렵다. 오버피팅의 가장 확실한 해결책은 **데이터를 늘리는 것**이다 (전체 55000장으로 학습한 3.6절의 network는 검증 정확도가 96% 였다).

4.3 Adam과 가중치 감소: AdamW

SGD에서는 "손실에 $\frac{\lambda}{2}\|W\|^2$를 더하는 것(L2 규제)"과 "매 단계 W를 $(1-\alpha\lambda)$배로 줄이는 것(가중치 감소)"이 같았다. 그러나 **Adam에서는 둘이 다르다.** L2 규제를 쓰면 λW가 gradient에 섞여 들어가 $\boldsymbol{m}, \boldsymbol{v}$를 거치므로

$$W \leftarrow W - \alpha \frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}} + \varepsilon}, \qquad \hat{\boldsymbol{m}},\ \hat{\boldsymbol{v}}\text{는 } \boldsymbol{g} = \frac{\partial L}{\partial W} + \lambda W\text{로부터 계산}$$

감소 항 λW도 성분마다 $\sqrt{\hat{\boldsymbol{v}}}$로 나누어진다. 그 결과 gradient가 큰 parameter는 거의 감소되지 않고, gradient가 작은 parameter는 지나치게 감소된다. **AdamW** (Loshchilov & Hutter, 2019)는 감소 항을 Adam의 갱신과 **분리(decouple)**하여 직접 적용한다.

$$\boxed{W \leftarrow W - \alpha\Big(\frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}} + \varepsilon} + \lambda W\Big), \qquad \hat{\boldsymbol{m}},\ \hat{\boldsymbol{v}}\text{는 } \boldsymbol{g} = \frac{\partial L}{\partial W}\text{만으로 계산}}$$

즉 모든 가중치가 매 단계 $(1-\alpha\lambda)$배씩 똑같은 비율로 줄어든다. 별도 심화 자료에서 소개하는 것처럼 AdamW는 Transformer와 대규모 언어모델 학습의 사실상 기본 optimizer이다.

`optimizers.py`에 `Adam`을 상속하여 `AdamW`를 추가하였다. 가중치를 먼저 줄이고, 나머지는 Adam과 똑같다 (PyTorch의 `torch.optim.AdamW`와 같은 순서).

```
class AdamW(Adam):
    """Adam + decoupled weight decay:  W <- W - lr * (m_hat / (sqrt(v_hat) + eps) + wd * W)
       The decay does NOT go through m and v.  Only weights (keys "W1", "W2", ...) are decayed."""
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-8, weight_decay=0.01):
        super().__init__(lr, beta1, beta2, eps)
        self.weight_decay = weight_decay

    def update(self, params, grads):
        for key in params:
            if key.startswith("W"):
                params[key] -= self.lr * self.weight_decay * params[key]   # shrink W
        super().update(params, grads)                      # the usual Adam step
```

AdamW를 쓸 때는 network에 `weight_decay_lambda`를 주지 않는다 (감소를 optimizer가 맡는다).

같은 오버피팅 실험을 Adam ($\alpha = 0.001$)으로 해 보면 다음과 같다.

```
runs = {
```

```
    "Adam":              (make_net(), Adam(lr=0.001)),
    "Adam + L2 (0.1)":   (make_net(weight_decay_lambda=0.1), Adam(lr=0.001)),
    "AdamW (wd = 0.1)": (make_net(), AdamW(lr=0.001, weight_decay=0.1)),
}
```

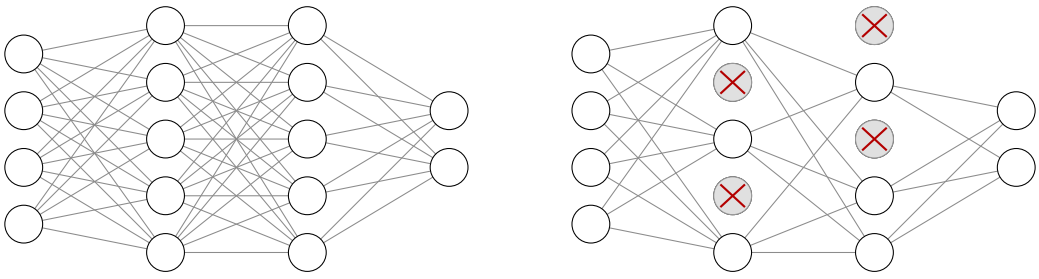
	훈련 정확도	시험 정확도	차이
Adam	100%	83.3%	16.7%p
Adam + L2 ($\lambda = 0.1$)	91.7%	72.9%	18.8%p
AdamW ($\lambda = 0.1$)	100%	83.1%	16.9%p

같은 $\lambda = 0.1$ 인데도 두 방법의 결과가 전혀 다르다. L2 규제는 $\sqrt{\hat{v}}$로 나누어지면서 gradient가 작은 가중치를 강하게 눌러 과소적합을 일으켰고, AdamW의 감소는 한 단계에 $\alpha\lambda = 10^{-4}$의 비율이라 900번의 갱신 동안 가중치가 모두 합쳐 $(1 - 10^{-4})^{900} \approx 0.91$배로만 줄어 영향이 작았다. 이처럼 **같은 숫자 λ라도 방법에 따라 의미가 다르므로**, 실무에서는 AdamW의 `weight_decay`를 0.01 ~ 0.1 정도에서 검증 데이터로 고르며, 수만 단계 이상의 긴 학습에서 효과가 드러난다.

4.4 드롭아웃

가중치 감소는 구현이 간단하고 어느 정도 효과가 있지만, network가 복잡해지면 가중치 감소만으로는 대응하기 어려워진다. 이럴 때 흔히 쓰는 방법이 **드롭아웃(dropout)**이다 (Srivastava, Hinton 등, 2014).

드롭아웃은 학습할 때 은닉층의 node를 **무작위로 골라 삭제**한다. 삭제된 node는 신호를 전달하지 않는다. 미니배치마다 삭제할 node를 새로 고르므로, 매 단계 서로 다른 (더 작은) network를 학습시키는 셈이다. 추론할 때는 모든 node를 사용한다.



(a) 일반 신경망 (b) 드롭아웃을 적용한 신경망

드롭아웃의 개념. (b)에서 × 표시된 은닉층 node는 이번 미니배치에서 삭제되었다.

순전파와 역전파. 삭제 비율을 p (`dropout_ratio`)라 하자. 학습할 때 은닉층의 출력 Z ($h \times m$)와 같은 크기의 **mask** M을 만들어, 각 원소를 확률 $1 - p$로 1 (통과), 확률 p로 0 (삭제)으로 정한 뒤 $Z \odot M$을 다음 층으로 보낸다. 역전파는 ReLU와 똑같다. 순전파에서 통과한 원소는 gradient를 그대로 통과시키고, 삭제된 원소는 gradient도 0이다.

$$\text{순전파: } Y = Z \odot M, \qquad \text{역전파: } \frac{\partial L}{\partial Z} = \frac{\partial L}{\partial Y} \odot M.$$

추론할 때의 크기 보정. 학습할 때는 평균적으로 $(1-p)$의 비율만 신호를 전달했으므로, 추론할 때 모든 node를 다 쓰면 다음 층으로 가는 신호가 $\frac{1}{1-p}$ 배로 커진다. 두 가지 보정 방법이 있다.

- **추론할 때 보정**: 학습할 때는 $Z \odot M$을 그대로 쓰고, 추론할 때 출력에 $(1-p)$를 곱한다.
- **학습할 때 보정 (inverted dropout)**: 학습할 때 살아남은 값을 $\frac{1}{1-p}$ 배로 키우고 ($Z \odot M/(1-p)$), 추론할 때는 **아무것도 하지 않는다**.

두 방법 모두 학습할 때 출력의 기댓값이 추론할 때의 출력과 같아지도록 맞춘 것이다. M은 확률 $1-p$로 1, 확률 p로 0이므로 $\mathbb{E}[M] = 1-p$이고, node의 값 z에 대하여

$$\begin{aligned} \text{추론할 때 보정:} \quad & \mathbb{E}[z\,M] = (1-p)\,z \quad &&= \text{추론할 때의 출력 } (1-p)\,z, \\ \text{학습할 때 보정 (inverted):} \quad & \mathbb{E}\Big[\frac{z\,M}{1-p}\Big] = z \quad &&= \text{추론할 때의 출력 } z. \end{aligned}$$

PyTorch의 `nn.Dropout`을 비롯한 프레임워크는 추론 코드를 건드리지 않아도 되는 inverted dropout을 쓴다. 이 강의의 코드도 inverted dropout으로 구현하였다.

```
class Dropout:
    """Inverted dropout (the same as PyTorch nn.Dropout)
       training: erase each node w.p. dropout_ratio, scale the survivors by 1/keep
       test:     use every node as it is"""
    def __init__(self, dropout_ratio=0.5):
        self.dropout_ratio = dropout_ratio
        self.mask = None

    def forward(self, X, train_flg=True):
        if train_flg:
            keep = 1.0 - self.dropout_ratio
            # *X.shape unpacks the shape tuple: rand(*(100, 50)) is rand(100, 50)
            self.mask = (np.random.rand(*X.shape) < keep) / keep   # 0 or 1/keep
            return X * self.mask
        return X

    def backward(self, dY):
        return dY * self.mask              # erased nodes pass no gradient
```

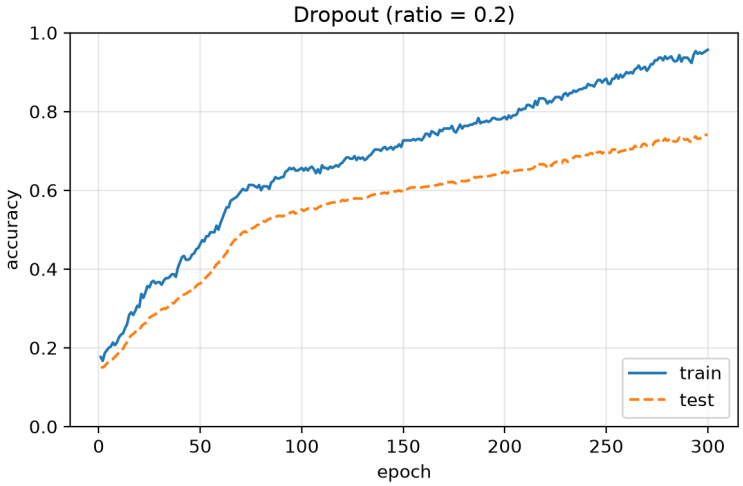
mask에 $\frac{1}{1-p}$이 이미 들어 있으므로 역전파도 `dY * self.mask` 한 줄이다. `MultiLayerNet`에서는 `use_dropout=True`이면 각 은닉층의 활성화 함수 뒤에 Dropout 계층을 넣는다.

```
                self.layers.append(Relu() if activation == "relu" else Sigmoid())
                if use_dropout:                          # ... -> activation -> Dropout
                    self.layers.append(Dropout(dropout_ratio))
```

Dropout 계층도 BatchNorm 계층처럼 `train_flg`에 따라 다르게 동작하므로, 3.5절의 `predict`가 두 계층에 `train_flg`를 넘긴다. 학습할 때 쓰는 `loss`는 `train_flg=True`, 평가할 때 쓰는 `accuracy`는 `train_flg=False`이다.

실험. 앞과 같은 조건에서 삭제 비율 $p = 0.2$로 드롭아웃을 적용한다.

```
# (3) dropout
acc = train(make_net(use_dropout=True, dropout_ratio=0.2), SGD(lr=0.01))
```



드롭아웃 ($p = 0.2$)을 적용했을 때의 정확도

	200 epoch		300 epoch	
	훈련	시험	훈련	시험
규제 없음	100%	80.3%	100%	80.7%
가중치 감소 ($\lambda = 0.1$)	85.3%	70.7%	85.7%	71.8%
드롭아웃 ($p = 0.2$)	78.7%	64.8%	95.7%	74.1%

- 드롭아웃을 쓰면 훈련 정확도가 훨씬 천천히 올라간다. 매 단계 일부 node가 사라지므로 network가 훈련 데이터를 **외우기 어렵기** 때문이다. 학습하는 동안 훈련과 시험 정확도의 차이는 규제가 없을 때보다 작다 (200 epoch에서 13.9%p).
- 그러나 학습을 계속하면 결국 훈련 정확도가 100%에 가까워지고 차이도 다시 벌어진다 (300 epoch에서 21.6%p). 드롭아웃은 오버피팅을 **늦추고 억제**하지만, 데이터 300장으로 13만 개의 parameter를 학습하는 상황 자체를 바꾸지는 못한다.
- 이 실험처럼 좁고 깊은 network에 데이터가 아주 적을 때는 드롭아웃의 효과가 크지 않다. 드롭아웃은 node가 많은 넓은 network를 충분한 데이터로 오래 학습할 때 효과적이다. 삭제 비율 p도 λ처럼 검증 데이터로 고르는 hyperparameter이다.

드롭아웃은 왜 효과가 있을까: 원리와 직관

(1) 공적응(co-adaptation)을 막는다. 드롭아웃을 처음 제안한 Hinton 등(2012)의 동기이다. 큰 network를 적은 데이터로 학습시키면 은닉 node들이 서로 짝을 지어 학습한다. "node A가 틀리는 부분을 node B가 바로잡아 준다"는 식의 조합이다. 이런 조합은 훈련 데이터에만 맞는 복잡한 규칙, 즉 훈련 데이터의 우연한 잡음까지 외운 결과인 경우가 많다. 드롭아웃으로 매번 무작위로 node를 지우면, 어떤 node도 특정 동료가 항상 있다고 기대할 수 없다. 그래서

- 각 node가 **혼자서도, 어떤 동료들과 함께여도 쓸모 있는 특징**을 배우고,
- 같은 정보가 **여러 node에 나누어 중복해서** 저장된다.

즉 추론을 특정 node에 지나치게 의존하지 않고 여러 node에 분산시킨다. 누가 결석할지 모르는 팀 프로젝트를 생각하면 된다. 매번 무작위로 몇 명이 빠지면 특정 사람에게만 맡겨 둘 수 없으므로 핵심 내용을 여러 사람이 나누어 알게 된다. Srivastava 등(2014)은 이것을 유성생식에 비유하였다. 유전자는 정해진 짝이 아니라 무작위로 섞인 다른 유전자들과 함께 여도 잘 작동해야 살아남는다.

(2) 앙상블 학습을 하나의 network로 흉내 낸다. **앙상블 학습(ensemble learning)**은 여러 network를 따로 학습시켜 추론할 때 그 출력의 평균을 내는 방법으로, 성능을 몇 % 개선하는 것으로 알려져 있다. 드롭아웃은 학습할 때마다 node를 무작위로 삭제하므로 매번 **다른 network**를 학습시키는 것으로 볼 수 있다. node가 n개면 가능한 network는 2^n개이고, 이들은 가중치를 공유한다. 추론할 때 모든 node를 쓰고 크기를 보정하는 것은 이 수많은 network의 출력을 (근사적으로) 평균 내는 효과를 낸다. 여러 사람이 각자 판단한 뒤 의견을 평균하면 한 사람의 들쭉날쭉한 실수가 줄어드는 것과 같다.

(3) 잡음을 넣는 규제이다. 학습할 때 은닉층에 잡음을 넣으므로, 은닉층에서 하는 데이터 증강으로 볼 수 있다. 입력이 조금 달라져도 같은 답을 내도록 학습되는 것이다. 선형회귀처럼 단순한 모델에서는 드롭아웃이 L2 규제(가중치 감소)와 비슷한 벌칙이 된다는 것도 알려져 있다 (Wager 등, 2013).

세 관점 모두 **훈련 데이터를 외우기 어렵게 만들어, 처음 보는 데이터에서도 잘 맞히게 한다**는 같은 목표를 향한다.

실무에서의 규제

- PyTorch에서는 `nn.Dropout(p)`를 쓰고, `model.train()`과 `model.eval()`로 학습과 추론을 전환한다. 가중치 감소는 optimizer의 인자로 준다.

```
torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.01)
```

  `torch.optim.SGD`와 `torch.optim.Adam`의 `weight_decay`는 L2 규제 방식이다.

- 배치 정규화를 쓰는 CNN에서는 드롭아웃을 마지막 fully-connected 층에만 쓰거나 아예 쓰지 않는 경우가 많다. 배치 정규화 자체에 규제 효과가 있기 때문이다 (3.6절).
- Transformer에서는 $p = 0.1$ 정도의 드롭아웃을 흔히 쓰지만, 데이터가 매우 많은 대규모 언어모델의 사전학습에서는 드롭아웃을 쓰지 않는 ($p = 0$) 경우도 많다. 데이터가 충분하면 오버피팅 자체가 잘 일어나지 않기 때문이다.
- 이 밖의 규제로 **데이터 증강(data augmentation)** (이미지를 회전, 이동, 반전하여 데이터를 늘림), **조기 종료(early stopping)** (검증 정확도가 더 오르지 않으면 학습을 멈춤) 등이 있다.

바른 학습을 위해: 정리

- **오버피팅**: 훈련 데이터에만 맞추어져 처음 보는 데이터에서 성능이 떨어지는 현상. 훈련 정확도와 시험(검증) 정확도의 차이로 확인한다.
- **가중치 감소**: 손실에 $\frac{\lambda}{2}\sum\|W\|^2$를 더한다. gradient에 λW를 더하면 된다. Adam에서는 감소를 분리한 **AdamW**를 쓴다.
- **드롭아웃**: 학습할 때 node를 확률 p로 무작위 삭제하고, 추론할 때는 모든 node를 쓴다.
- 규제의 세기 (λ, p)는 hyperparameter이며, **검증 데이터**로 고른다. 너무 강하면 과소적합이 된다.

5 적절한 하이퍼파라미터 값 찾기

신경망에는 가중치 W와 bias B 말고도 사람이 정해 주어야 하는 값이 많다. 이런 값을 **하이퍼파라미터(hyperparameter)**라고 한다.

	매개변수 (parameter)	하이퍼파라미터 (hyperparameter)
예	W, B, 배치 정규화의 $\boldsymbol{\gamma}$, $\boldsymbol{\beta}$	학습률 α, 가중치 감소 λ, 드롭아웃 비율 p, 미니배치 크기, epoch 수, 층의 수, 각 층의 node 수, 초깃값의 표준편차, optimizer의 $\beta_1, \beta_2, \ldots$
정하는 방법	훈련 데이터의 손실을 줄이도록 **gradient로 자동** 학습	학습 **전에** 사람이 정한다. 여러 값을 시도하여 비교한다.

지금까지의 실험에서도 학습률을 0.01로 할지 0.1로 할지, λ를 얼마로 할지 계속 골라야 했다.

하이퍼파라미터를 잘못 고르면 같은 network라도 성능이 크게 떨어진다 (1.9절의 SGD, 4.2절의 $\lambda = 0.3$). 이 절에서는 하이퍼파라미터를 **무엇을 기준으로, 어떻게** 고르는지 다룬다.

5.1 검증 데이터

지금까지 우리는 데이터를 크게 두 그룹으로 나누어 생각했다.

- **훈련 데이터(training data)**: 매개변수 W, B를 학습하는 데 쓴다.
- **시험 데이터(test data)**: 학습이 끝난 network의 **범용 성능**, 즉 처음 보는 데이터에 대한 성능을 평가하는 데 쓴다.

그렇다면 하이퍼파라미터의 좋고 나쁨은 무엇으로 평가해야 할까?

시험 데이터로 하이퍼파라미터를 고르면 안 된다. 여러 하이퍼파라미터로 학습시킨 뒤 시험 정확도가 가장 높은 것을 고른다고 하자. 그러면 하이퍼파라미터가 **시험 데이터에 맞추어** 선택된다. 시험 데이터에도 우연한 특징이 있으므로, 그 우연에 잘 맞는 하이퍼파라미터가 뽑히게 되고, 이렇게 고른 network의 시험 정확도는 처음 보는 데이터에 대한 실제 성능보다 **높게 나온다**. 매개변수가 훈련 데이터에 오버피팅하듯이, 하이퍼파라미터가 시험 데이터에 오버피팅하는 것이다. 그러면 시험 데이터는 더 이상 "처음 보는 데이터"의 역할을 할 수 없다.

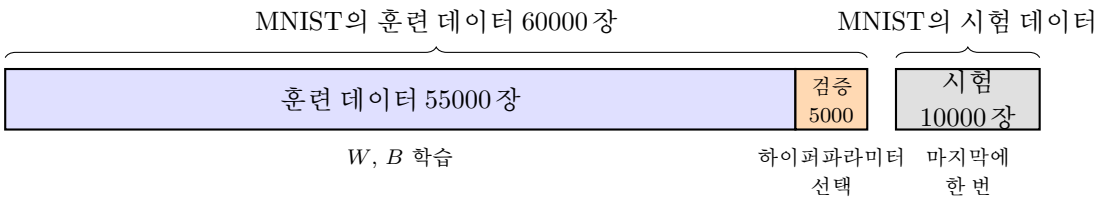
그래서 하이퍼파라미터를 고르기 위한 데이터를 따로 둔다. 이것을 **검증 데이터(validation data)**라고 한다.

훈련 / 검증 / 시험 데이터

- **훈련 데이터**: 매개변수 (W, B, γ, β) 학습
- **검증 데이터**: 하이퍼파라미터의 성능 평가와 선택
- **시험 데이터**: 최종 선택한 network의 범용 성능 평가. **마지막에 단 한 번**만 쓴다.

시험 공부에 비유하면 훈련 데이터는 교재의 연습문제, 검증 데이터는 공부 방법을 점검하는 모의고사, 시험 데이터는 수능이다. 수능 문제를 미리 보고 공부 방법을 정하면 수능 점수는 실력을 나타내지 못한다.

검증 데이터는 훈련 데이터에서 떼어 낸다. MNIST처럼 훈련 데이터와 시험 데이터만 나누어져 있는 데이터셋이 많다. 이때는 훈련 데이터의 일부(보통 10 ~ 20%)를 무작위로 골라 검증 데이터로 떼어 두고, 나머지만 학습에 쓴다.



이 강의의 `mnist_data.py`가 바로 이렇게 나눈다. 지난주의 `mnist_mlp.py` 과제에서 간단히 소개했던 분할과 같은 것이다.

```
    # hold out n_val training images as a VALIDATION set
    perm = np.random.permutation(x_all.shape[0])
    val_idx, train_idx = perm[:n_val], perm[n_val:]
    return (x_all[train_idx], t_all[train_idx],
            x_all[val_idx], t_all[val_idx],
            x_test, t_test)
```

이 노트의 1.9, 2.4, 3.6절에서 optimizer, 초깃값, 배치 정규화를 비교할 때 시험 정확도가 아니라 **검증 정확도**를 보고한 것도 이 때문이다. 여러 방법 중 무엇이 나은지 **고르는** 실험이었기 때문이다. (4.1절에서 시험 데이터를 쓴 것은 고르기 위해서가 아니라 오버피팅 현상 자체를 보여 주기 위해서였다.)

데이터가 적을 때: 교차 검증

데이터가 아주 적으면 검증 데이터를 떼어 내기 아깝고, 검증 데이터가 작으면 검증 정확도 자체가 우연에 크게 흔들린다. 이럴 때는 훈련 데이터를 K개 묶음으로 나누어, 한 묶음을 검증 데이터로, 나머지를 훈련 데이터로 쓰는 실험을 K번 반복하고 검증 정확도를 평균하는 **K-겹 교차 검증(K-fold cross validation)**을 쓴다. 비용이 K배이므로 데이터가 많은 딥러닝에서는 보통 하나의 검증 데이터만 둔다.

5.2 하이퍼파라미터 최적화

하이퍼파라미터는 gradient로 학습할 수 없으므로, 값을 바꾸어 가며 직접 학습시켜 보고 검증 정확도를 비교하는 수밖에 없다. 핵심은 **좋은 값이 있을 법한 범위를 점점 좁혀 가는 것**이다.

하이퍼파라미터 최적화의 절차

0단계 하이퍼파라미터의 범위를 대략적으로 정한다 (예: 학습률 $10^{-5} \sim 1$).

1단계 그 범위에서 하이퍼파라미터 값을 **무작위로** 뽑는다.

2단계 뽑은 값으로 학습하고 **검증 데이터**로 정확도를 평가한다. 이때 epoch은 작게 한다.

3단계 1, 2단계를 일정 횟수 (예: 100번) 반복하고, 정확도가 좋은 값들이 모인 곳으로 범위를 좁힌다.

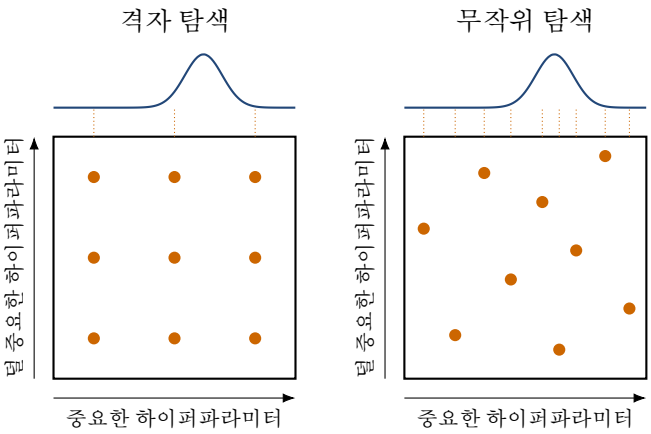
범위가 충분히 좁아지면 그 안에서 하나를 골라, 전체 데이터로 오래 학습한 뒤 시험 데이터로 최종 평가한다.

로그 눈금(log scale)으로 뽑는다. 학습률이 0.001인지 0.002인지보다 0.001인지 0.01인지가 훨씬 중요하다. 하이퍼파라미터는 대개 **몇 배**가 차이 나는지가 중요하므로, 지수를 균등하게 뽑는다.

$$\alpha = 10^{u}, \qquad u \sim \mathrm{Uniform}(-5,\, 0)$$

α를 10^{-5}과 1 사이에서 그냥 균등하게 뽑으면 90%가 0.1보다 큰 값이 되어 작은 학습률은 거의 시험해 보지 못한다.

격자 탐색보다 무작위 탐색. 값을 $\{10^{-4}, 10^{-3}, 10^{-2}\}$처럼 격자로 정해 모든 조합을 시험하는 **격자 탐색(grid search)**보다, 무작위로 뽑는 **무작위 탐색(random search)**이 대개 더 좋은 결과를 낸다 (Bergstra & Bengio, 2012). 하이퍼파라미터들 중 결과에 큰 영향을 주는 것은 일부뿐이기 때문이다. 아래 그림에서 가로축이 중요한 하이퍼파라미터, 세로축이 거의 영향이 없는 하이퍼파라미터라고 하자. 똑같이 9번 시험해도 격자 탐색은 중요한 축에서 3가지 값만 보지만, 무작위 탐색은 9가지 값을 본다.



위의 곡선은 중요한 하이퍼파라미터에 따른 성능. 무작위 탐색은 봉우리 근처의 값을 시험할 기회가 많다.

시간을 줄이는 방법. 하이퍼파라미터 하나를 평가하려면 network를 한 번 학습시켜야 하므로 시간이 많이 걸린다. 그래서 탐색 단계에서는 **데이터의 일부**와 **적은 epoch**으로 학습시켜 가망이 없는 값을 빨리 걸러 낸다. 좋은 범위를 찾은 뒤에 전체 데이터로 오래 학습한다.

5.3 구현: 학습률과 가중치 감소의 무작위 탐색

`hyperparameter_search.py`에서는 책에서와 같이 학습률 α와 가중치 감소 λ 두 가지를 무작위 탐색으로 찾는다. 시간을 줄이려고 훈련 데이터는 500장만 쓰고, 50 epoch만 학습한다. network는 4.1절과 같은 7층 network (은닉층 6개, 각 100개 node, He 초깃값, SGD)이다. 평가는 검증 데이터 5000장으로 한다.

```
x_train, t_train, x_val, t_val, x_test, t_test = load_mnist_split()   # 55000 / 5000 / 10000
x_train, t_train = x_train[:500], t_train[:500]   # a small training set -> fast trials
```

```
        train_acc.append(net.accuracy(x_train, t_train))
        val_acc.append(net.accuracy(x_val, t_val))   # VALIDATION data, not test data
```

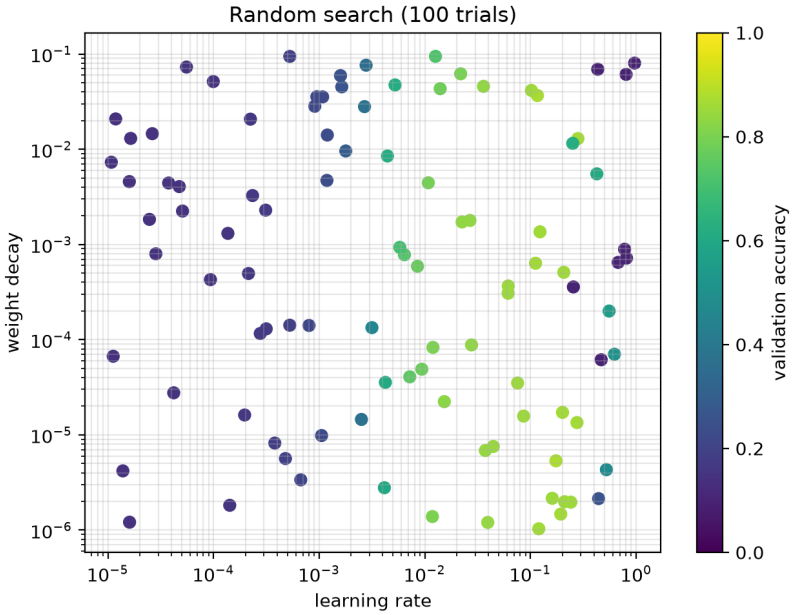
학습률은 $10^{-5} \sim 1$, 가중치 감소는 $10^{-6} \sim 10^{-1}$의 범위에서 로그 눈금으로 100번 뽑는다.

```
n_trials = 100
rng = np.random.default_rng(0)                    # separate random numbers for the sampling
results = []
for trial in range(n_trials):
    lr = 10 ** rng.uniform(-5, 0)                 # 1e-5 ~ 1
    weight_decay = 10 ** rng.uniform(-6, -1)      # 1e-6 ~ 0.1
```

```
    _, train_acc, val_acc = train(lr, weight_decay)
    results.append({"lr": lr, "wd": weight_decay, "train": train_acc, "val": val_acc})
```

모든 시행에서 초깃값과 미니배치 순서는 같게 하여 하이퍼파라미터의 차이만 비교한다.

결과. 100번의 시행을 학습률과 가중치 감소의 평면에 찍고, 검증 정확도로 색을 칠하면 다음과 같다.



무작위 탐색 100번의 결과. 점 하나가 하나의 시행이고, 색이 밝을수록 검증 정확도가 높다.

학습률 구간별로 정리하면 다음과 같다.

```
print("======= by learning rate =======")
edges = [1e-5, 1e-3, 1e-2, 3e-2, 1e-1, 3e-1, 1.0]
for lo, hi in zip(edges[:-1], edges[1:]):
    accs = [r["val"][-1] for r in results if lo <= r["lr"] < hi]
    print(f"lr {lo:.0e} ~ {hi:.0e}: {len(accs):3d} trials  best {max(accs):.4f}  mean {
    np.mean(accs):.4f}")
```

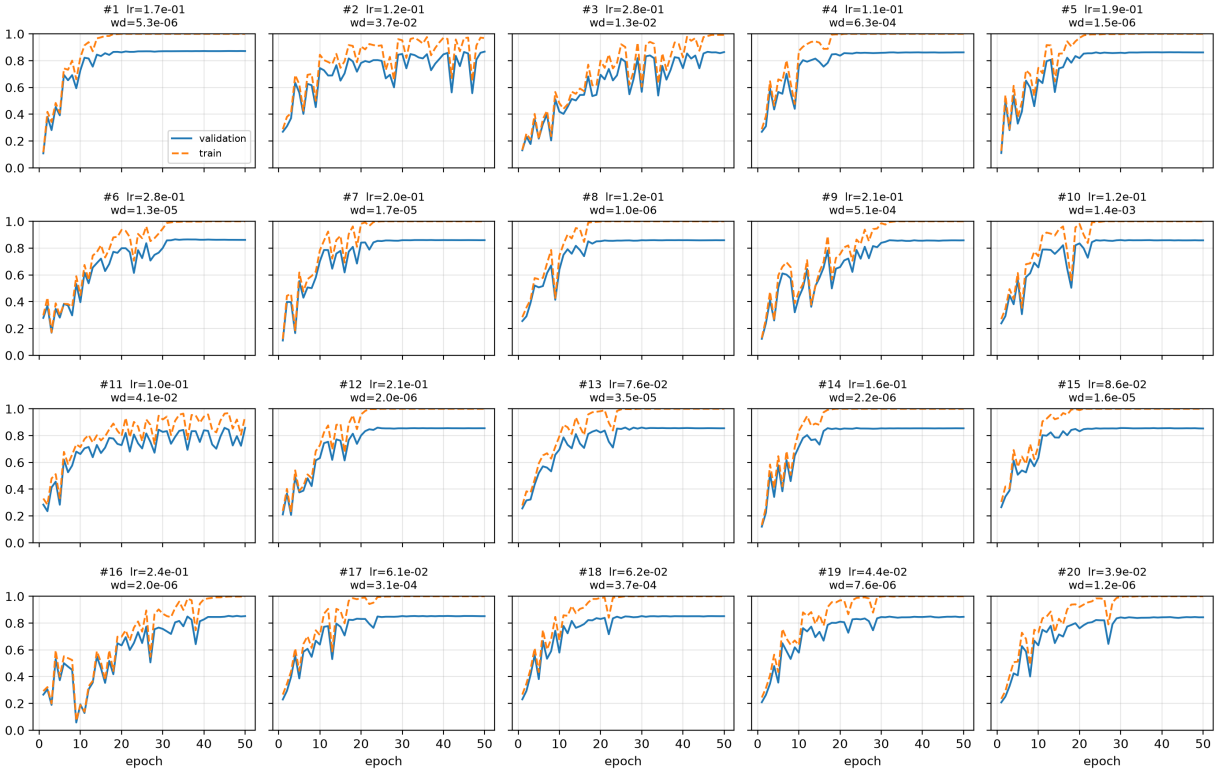
학습률	$10^{-5} \sim 10^{-3}$	$10^{-3} \sim 10^{-2}$	$0.01 \sim 0.03$	$0.03 \sim 0.1$	$0.1 \sim 0.3$	$0.3 \sim 1$
시행 수	34	20	10	8	16	12
최고 검증 정확도	22.0%	77.6%	83.5%	85.5%	**87.1%**	60.6%
평균 검증 정확도	16.3%	47.2%	80.2%	**84.7%**	79.8%	25.7%

- **학습률이 결과를 거의 결정한다.** 그림의 색이 가로 방향(학습률)으로는 크게 변하지만 세로 방향(가중치 감소)으로는 거의 변하지 않는다. 학습률이 10^{-3}보다 작으면 50 epoch 동안 거의 학습되지 않고, 0.3보다 크면 학습이 발산하여 정확도가 10% 근처로 떨어진다.
- **가중치 감소는 $10^{-6} \sim 10^{-2}$ 범위에서 거의 영향이 없다.** 이 범위에서는 감소량이 학습률에 비해 너무 작기 때문이다. λ가 5×10^{-2} 이상이 되어야 정확도가 조금씩 떨어지기 시작한다. 앞에서

본 격자 탐색과 무작위 탐색의 그림에서 "덜 중요한 하이퍼파라미터"가 바로 이런 경우이다. 무작위 탐색 덕분에 100번의 시행이 모두 서로 다른 학습률을 시험하였다.

- 지금까지 SGD의 학습률로 0.01을 써 왔는데, 이 network에는 0.1 근처가 더 좋다. 하이퍼파라미터의 좋은 값은 network와 데이터에 따라 다르므로 매번 찾아야 한다.

검증 정확도가 높은 상위 20개 시행의 학습 곡선은 다음과 같다.



검증 정확도 상위 20개 시행의 학습 곡선 (실선: 검증, 점선: 훈련)

순위	검증 정확도	훈련 정확도	학습률	가중치 감소
1	87.1%	100%	1.7×10^{-1}	5.3×10^{-6}
2	86.7%	96.8%	1.2×10^{-1}	3.7×10^{-2}
3	86.4%	99.2%	2.8×10^{-1}	1.3×10^{-2}
4	86.2%	100%	1.1×10^{-1}	6.3×10^{-4}
5	86.1%	100%	1.9×10^{-1}	1.5×10^{-6}

- 상위 10개는 모두 학습률이 0.11 ~ 0.28이지만, 가중치 감소는 10^{-6}부터 4×10^{-2}까지 흩어져 있다.
- 학습률이 큰 쪽 (2위, 3위)은 학습 곡선이 크게 흔들린다. 마지막 epoch의 정확도가 우연히 높게 나왔을 수도 있다.
- 검증 데이터가 5000장이면 정확도 87%의 표준오차는 $\sqrt{0.87 \times 0.13/5000} \approx 0.5\%$p이다. 상위권의 1%p 정도의 차이는 우연과 구별하기 어렵다. 그래서 1등 하나를 고르기보다 **좋은 값들이 모여 있는 범위**를 찾는 것이 중요하다.

범위 좁히기. 이 결과로부터 다음 탐색의 범위를 학습률 0.03 ∼ 0.3, 가중치 감소 10^{-6} ∼ 10^{-2} 정도로 좁힐 수 있다. 좁힌 범위에서 다시 무작위 탐색을 하고, 필요하면 훈련 데이터와 epoch을 늘려 가며 이 과정을 반복한다.

마지막에 단 한 번: 시험 데이터. 하이퍼파라미터를 정한 뒤에야 비로소 시험 데이터를 쓴다.

```
best = results[0]
net, _, _ = train(best["lr"], best["wd"])
print(f"chosen: lr {best['lr']:.2e}, weight decay {best['wd']:.2e}  ->  "
      f"validation {best['val'][-1]:.4f}, test {net.accuracy(x_test, t_test):.4f}")
```

```
chosen: lr 1.74e-01, weight decay 5.34e-06  ->  validation 0.8714, test 0.8701
```

검증 정확도 87.1%와 시험 정확도 87.0%가 거의 같다. 여기서는 차이가 작았지만, 일반적으로 100개 중 검증 정확도가 **가장 높은** 것을 골랐으므로 검증 정확도는 실제 성능보다 약간 높게 나오는 경향이 있다. 그래서 최종 성능은 반드시 선택에 쓰지 않은 시험 데이터로 보고한다. 이 시험 정확도를 본 뒤에 다시 하이퍼파라미터를 바꾸면 시험 데이터도 "검증 데이터"가 되어 버린다.

실무에서의 하이퍼파라미터 탐색

- 가장 먼저, 가장 꼼꼼하게 찾아야 하는 것은 **학습률**이다. 그다음이 미니배치 크기, 가중치 감소, 드롭아웃 비율, network의 크기 등이다.
- 이전 시행의 결과를 보고 다음에 시험할 값을 똑똑하게 고르는 **베이즈 최적화(Bayesian optimization)**, 가망 없는 시행을 일찍 중단하여 시간을 아끼는 **Hyperband** 같은 방법이 있다. Python에서는 `Optuna` 같은 라이브러리로 쉽게 쓸 수 있다.
- 대규모 모델은 한 번 학습하는 데 며칠이 걸리므로 탐색을 많이 할 수 없다. 그래서 작은 모델에서 찾은 하이퍼파라미터를 큰 모델로 옮기는 방법, 논문과 공개된 설정에서 출발하는 방법을 많이 쓴다.
- 검증 정확도가 더 오르지 않을 때 학습을 멈추는 **조기 종료**에서도 epoch 수라는 하이퍼파라미터를 검증 데이터로 정하는 것이다.

하이퍼파라미터 찾기: 정리

- 하이퍼파라미터는 **검증 데이터**로 고른다. 시험 데이터는 마지막에 한 번만 쓴다.
- 검증 데이터는 훈련 데이터의 일부를 떼어 만든다 (MNIST: 55000 / 5000 / 10000).
- 범위를 넓게 잡고, **로그 눈금**으로 **무작위** 탐색하며, 좋은 값이 모인 곳으로 범위를 좁힌다.
- 탐색 단계에서는 데이터와 epoch을 줄여 시간을 아낀다.

6 연습문제

1. $f(x, y) = \frac{1}{20}x^2 + y^2$에서 SGD가 발산하지 않는 학습률 α의 범위를 구하라. 또 이 범위 안에서 x 방향의 수렴이 가장 빠른 α와 그때 x가 한 단계에 몇 배가 되는지 구하라.

2. 모멘텀에서 gradient가 매 단계 같은 값 $\boldsymbol{g}$로 일정하다고 하자. $\boldsymbol{v}_0 = \mathbf{0}$일 때 $\boldsymbol{v}_k$를 구하고, $k \to \infty$일 때 $\boldsymbol{v}_k \to -\dfrac{\alpha}{1-\beta}\boldsymbol{g}$임을 보이라.

3. 시작점 $(-7, 2)$에서 AdaGrad ($\alpha = 1.5$)로 두 번째 단계 (x_2, y_2)를 손으로 계산하라 (ε은 무시한다). 표의 값 $(-4.573,\ 0.136)$과 비교하라.

4. Adam에서 편향 보정을 하지 않으면 첫 단계의 이동량은 얼마가 되는가? ($\beta_1 = 0.9$, $\beta_2 = 0.999$, ε은 무시한다.) 편향 보정이 왜 필요한지 설명하라.

5. `mnist_optimizer_compare.py`에서 SGD의 학습률을 0.01에서 0.05, 0.1로 바꾸어 보라. SGD도 plateau를 빠져나오는가? 같은 조건에서 RMSProp ($\alpha = 0.001$)을 `optimizers`에 추가하여 네 기법과 비교하라.

6. 은닉층의 모든 가중치를 같은 값 c로 초기화하면 학습이 진행되어도 은닉층의 node들이 계속 같은 값을 가지는 이유를 역전파 식 $\frac{\partial L}{\partial W} = X^{\mathsf{T}}\frac{\partial L}{\partial A}$를 이용하여 설명하라.

7. $a = \sum_{i=1}^{n} z_i w_i$에서 $z_i = \max(0, a_i')$이고 a_i'이 평균 0인 대칭 분포를 따를 때 $\mathbb{E}[z_i^2] = \frac{1}{2}\mathrm{Var}(a_i')$임을 보이고, 이로부터 He 초깃값 $\mathrm{Var}(w) = \frac{2}{n}$을 유도하라.

8. `weight_init_activation.py`에서 층의 수를 5에서 20으로 늘려 ReLU + Xavier와 ReLU + He의 마지막 층 활성화값의 표준편차를 비교하라. Xavier에서 표준편차가 층마다 약 몇 배씩 줄어드는가?

9. 배치 정규화의 순전파 식 (1)–(4)를 보고, 같은 미니배치에서 Affine 계층의 가중치 W를 a배 ($a > 0$) 하면 BatchNorm 계층의 출력 Y가 변하지 않음을 보이라 (ε은 무시한다). 이 사실로부터 $\frac{\partial L}{\partial (aW)} = \frac{1}{a}\frac{\partial L}{\partial W}$임을 설명하라.

10. 3.3절의 단계 ②–⑦을 모두 대입하여 remarkbox의 식 $\frac{\partial L}{\partial x_i} = \frac{1}{\sqrt{\sigma_B^2+\varepsilon}}\left(g_i - \bar{g} - \hat{x}_i\,\overline{g\hat{x}}\right)$을 유도하라. ($\bar{g}$, $\overline{g\hat{x}}$는 각각 g_k, $g_k\hat{x}_k$의 미니배치 평균이다.)

11. `mnist_batchnorm.py`의 실험 2에서 `accuracy`가 이동평균 대신 미니배치 통계를 쓰도록 (`train_flg=True`) 바꾸고, 검증 데이터를 1장씩 넣어 정확도를 계산해 보라. 어떤 일이 생기는가? 왜 그런가?

12. 가중치 감소를 적용한 SGD의 갱신식이 $W \leftarrow (1-\alpha\lambda)W - \alpha\frac{\partial L}{\partial W}$임을 보이라. $\alpha = 0.01$, $\lambda = 0.1$일 때 gradient가 0이라면 W가 처음 값의 절반이 되기까지 몇 번의 갱신이 필요한가?

13. 하나의 parameter w에 대하여 gradient가 매 단계 g로 일정하다고 하자 ($g \neq 0$, ε은 무시). Adam에 L2 규제를 쓰면 감소 항 λw가 실제 갱신량에 주는 영향이 $|g|$에 따라 어떻게 달라지는지, AdamW와 비교하여 설명하라.

14. `mnist_overfit.py`에서 은닉층을 $[100]\times 6$ 대신 $[500, 500]$으로 바꾸고, 드롭아웃의 삭제 비율 $p = 0,\ 0.2,\ 0.5$를 **검증 데이터**로 비교하라. 어떤 p를 고르겠는가? 그 p로 학습한 network의 시험 정확도를 보고하라.

15. 하이퍼파라미터 10개를 시험하여 시험 정확도가 가장 높은 것을 골랐더니 시험 정확도가 98%였다. 이 98%를 처음 보는 데이터에 대한 성능이라고 보고해도 되는가? 왜 그런가? 어떻게 해야 하는가?

16. 학습률 α를 $[10^{-5},\ 1]$에서 (a) 그냥 균등하게, (b) $\alpha = 10^{u}$, $u \sim \mathrm{Uniform}(-5, 0)$으로 뽑을 때, $\alpha < 10^{-2}$일 확률을 각각 구하라.

17. `hyperparameter_search.py`의 결과를 보고 학습률과 가중치 감소의 범위를 좁혀 (100번 대신 30번) 다시 탐색하라. 좁힌 범위와 그 근거, 가장 좋은 검증 정확도를 표로 정리하라.